



Data Warehousing and
Business
Intelligence(IT3021)

Assignment 1

Registration Number – IT23191556

Name – Perera M.S

Group – Y3S2.01.02

Table of Contents

Task 1: Dataset Selection	3
Task 2: Preparation of Data Sources	5
Task 3: Solution Architecture	11
Task 4: Data Warehouse Design & Development	14
Task 5: ETL Development	16
Task 6: ETL Development – Accumulating Fact Tables	42

Task 1: Dataset Selection

1.1 Data Description

The selected dataset is a Parking Management OLTP dataset obtained from Kaggle. This dataset represents real-world parking transactions generated from parking systems such as parking meters and mobile-based applications.

The dataset contains approximately one million records, where each record corresponds to a single parking transaction. It includes key attributes such as Transaction ID, start time, end time, duration (in minutes), amount paid, kiosk ID, application zone ID, zone group, source, and payment method .

The data spans from 2019 to 2025, providing several years of continuous transactional data. This enables time-based analysis, including daily, monthly, and yearly trends in parking usage and revenue .

Additionally, the dataset includes data from multiple sources, primarily parking meters and mobile applications, along with various payment methods such as card payments and digital wallets. The structure of the dataset reflects a typical OLTP system, where data is recorded for each individual transaction in real time.

Original dataset link → <https://www.kaggle.com/code/muhammedaliyilmazz/parking-data-analysis-with-ml-dl-models/input>

1.2 Justification for Selection

This dataset was selected as it fulfills all the requirements for data warehouse development and analytical processing.

Firstly, it is a true OLTP dataset, containing detailed transactional data rather than pre-aggregated or summarized information.

Secondly, the dataset has a large volume of data (approximately one million records) and spans multiple years, ensuring sufficient data for analysis, aggregation, and reporting.

Furthermore, the dataset supports the design of a data warehouse schema, where parking transactions can be modeled as a fact table, and attributes such as time, location, and payment method can be modeled as dimension tables.

This dataset follows an OLTP structure because it records real-time transactional events, where each row represents a single parking transaction with detailed attributes. The data is normalized and operational in nature, supporting day-to-day operations rather than analytical aggregation.

1.3 ER Diagram

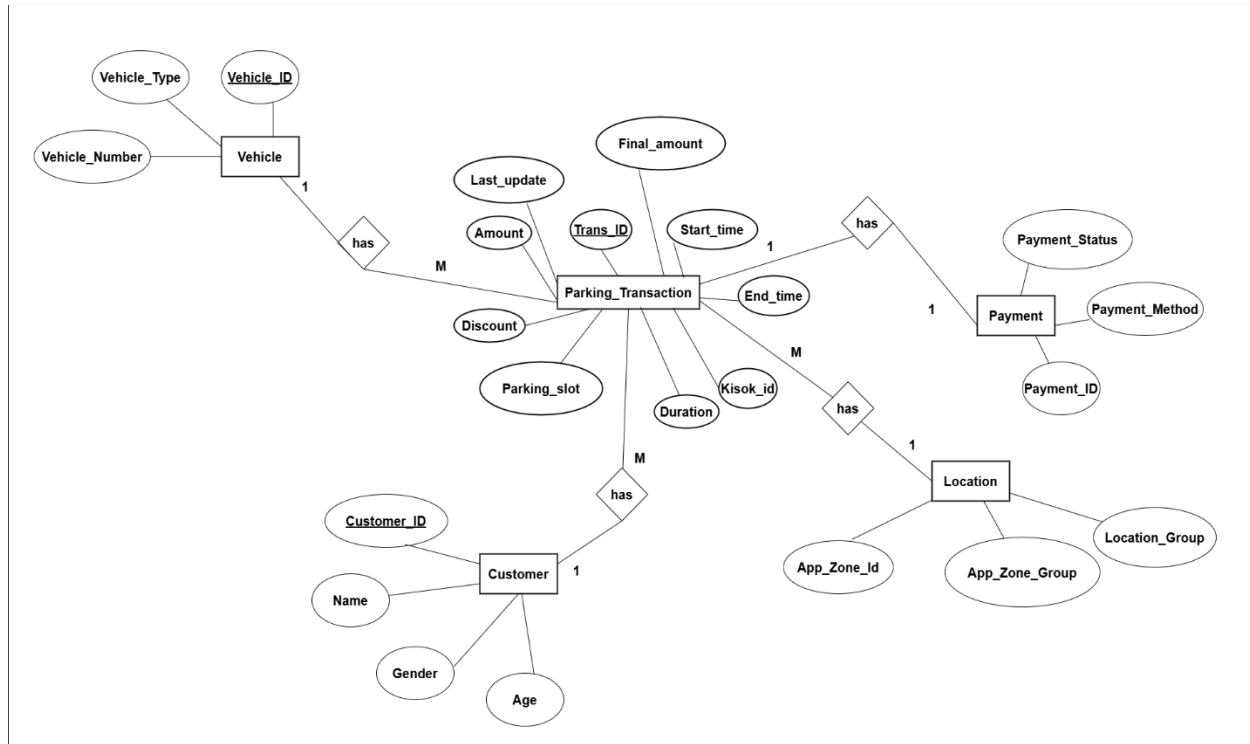


Figure: Entity-Relationship Diagram – Smart Parking Management System

Each entity contains a primary key (e.g., Customer_ID, Vehicle_ID), and foreign keys are used in Parking_Transaction to maintain relationships with Customer, Vehicle, Location, and Payment entities.

Key Relationships:

- Each Parking_Transaction is associated with one Customer, one Vehicle, one App Zone, and one Payment.
- Each Customer can have multiple Vehicles and multiple Parking Transactions.
- App Zones are grouped under App Zone Groups which belong to Location Groups, forming a three-level location hierarchy.
- Each Parking Transaction references one Payment record that describes the method and status of the payment.

Task 2: Preparation of Data Sources

2.1 Overview

To satisfy the requirement of using at least two distinct types of data sources, the Smart Parking dataset was organized into a combination of CSV files and Excel (.xlsx) files. This separation mirrors real-world scenarios where operational data originates from different systems and formats.

File Name	Format	Information Available
Parking_Transaction.csv	CSV	Transaction ID, Source, Duration , Start Time, End Time, Amount, Discount, Final Amount, Customer ID, Kiosk ID, App Zone, Payment ID, Last Update, Parking Slot
Payments.csv	CSV	Payment ID, Payment Method , Payment Status
Locations.xlsx	Excel	App Zone ID, App Zone Group, Location Group – forms the three-level location hierarchy
Vehicles.xlsx	Excel	Vehicle ID, Vehicle Type, Vehicle Number, Customer ID
Customers.xlsx	Excel	Customer ID, Customer Name, Gender, Age, Member Type

2.2 Source Type Classification

CSV Files (Flat File Sources)

The two CSV files – Parking_Transaction.csv and Payments.csv – represent the high-volume transactional data. These files are read using the Flat File Source component in SSIS.

- Parking_Transaction.csv: Contains the core parking event records. This is the primary fact data source with measures such as Amount, Discount, and Final_Amount, along with foreign keys linking to customers, payments, and zones.
- Payments.csv: Contains payment method and status information. This is joined to the transaction data to enrich the payment dimension.

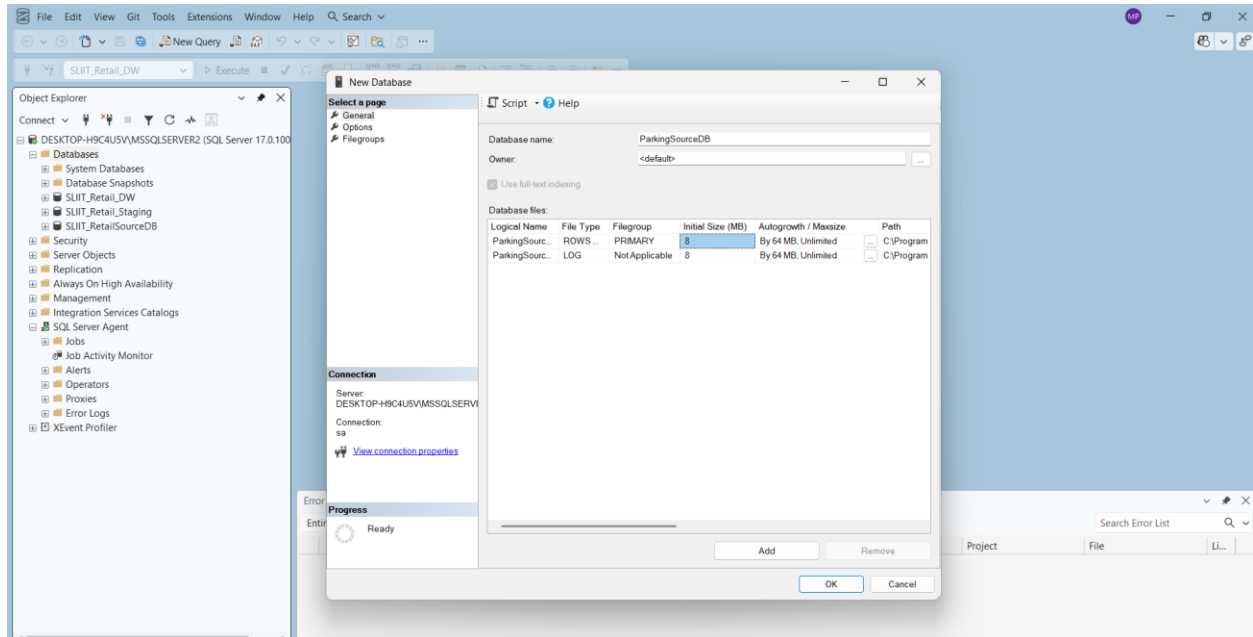
Excel Files (Excel Source)

The three Excel files – Customers.xlsx, Vehicles.xlsx, and Locations.xlsx – contain relatively slower-changing reference and master data. These files are read using the Excel Source component in SSIS.

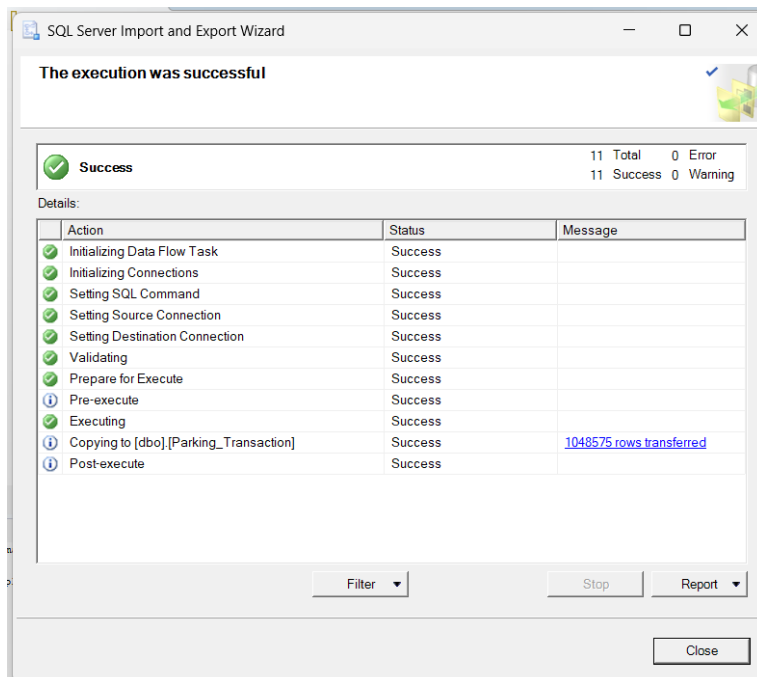
- Customers.xlsx: Customer master data used to populate the DimCustomer dimension table. The Member_Type attribute introduces a slowly changing attribute.
- Vehicles.xlsx: Vehicle master data linked to customers. Used to populate the DimVehicle dimension.
- Locations.xlsx: Zone and location hierarchy data used to populate DimLocation with a three-level hierarchy (Location Group > App Zone Group > App Zone).

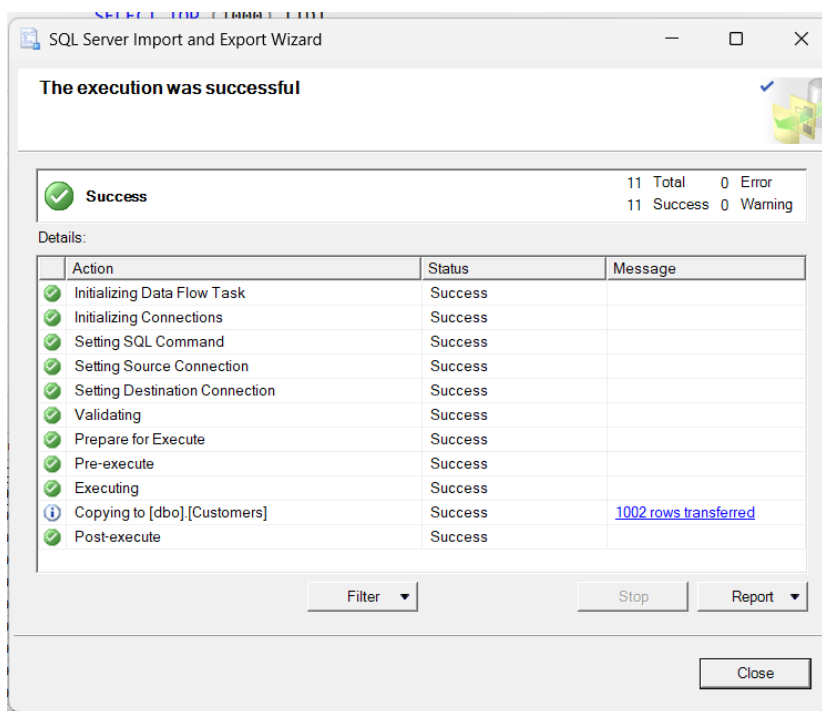
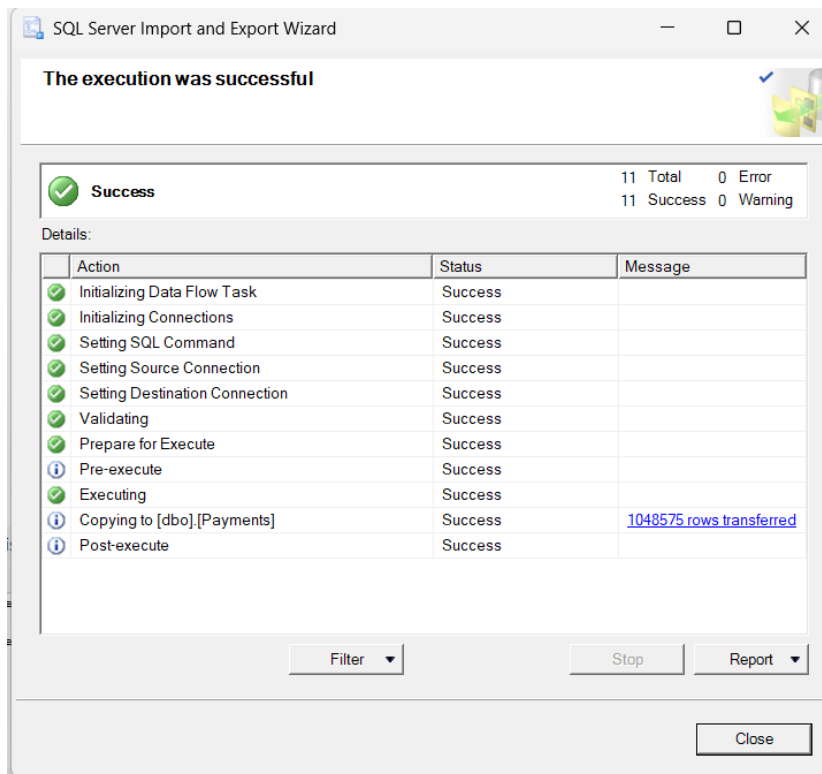
2.3 Source Database Creation(ParkingSourceDB)

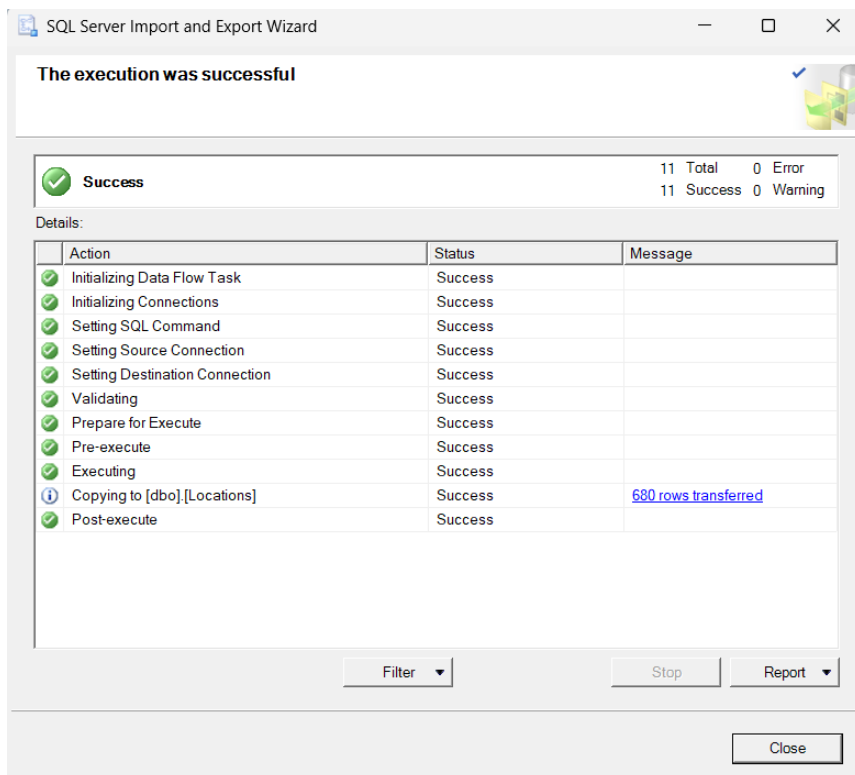
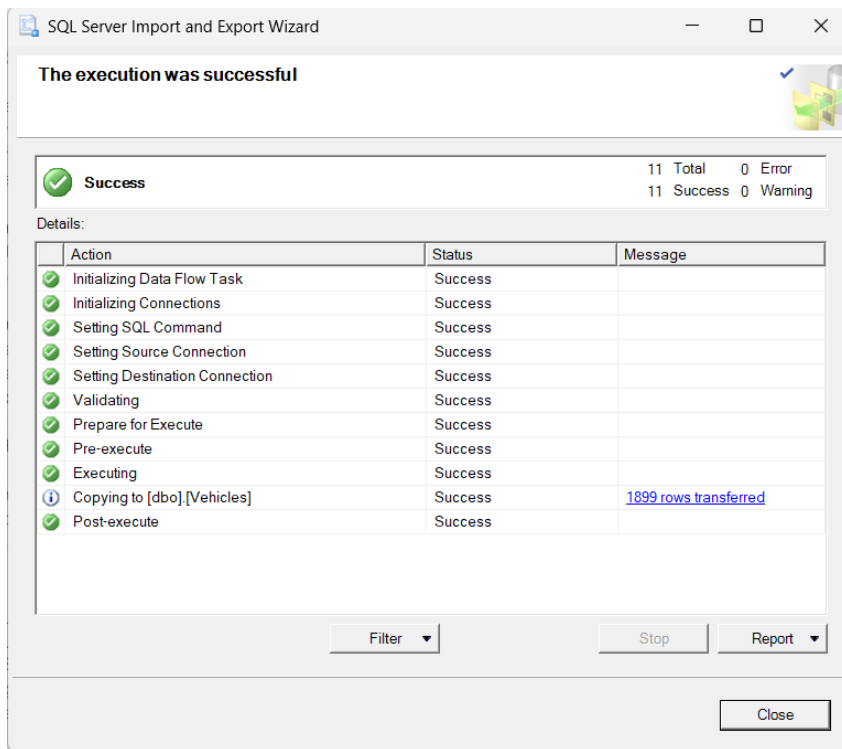
All five source files were loaded into a source database named ParkingSourceDB in SQL Server to centralise the raw data before staging. The SQL Server Import and Export Wizard was used to import each file into its respective staging table.



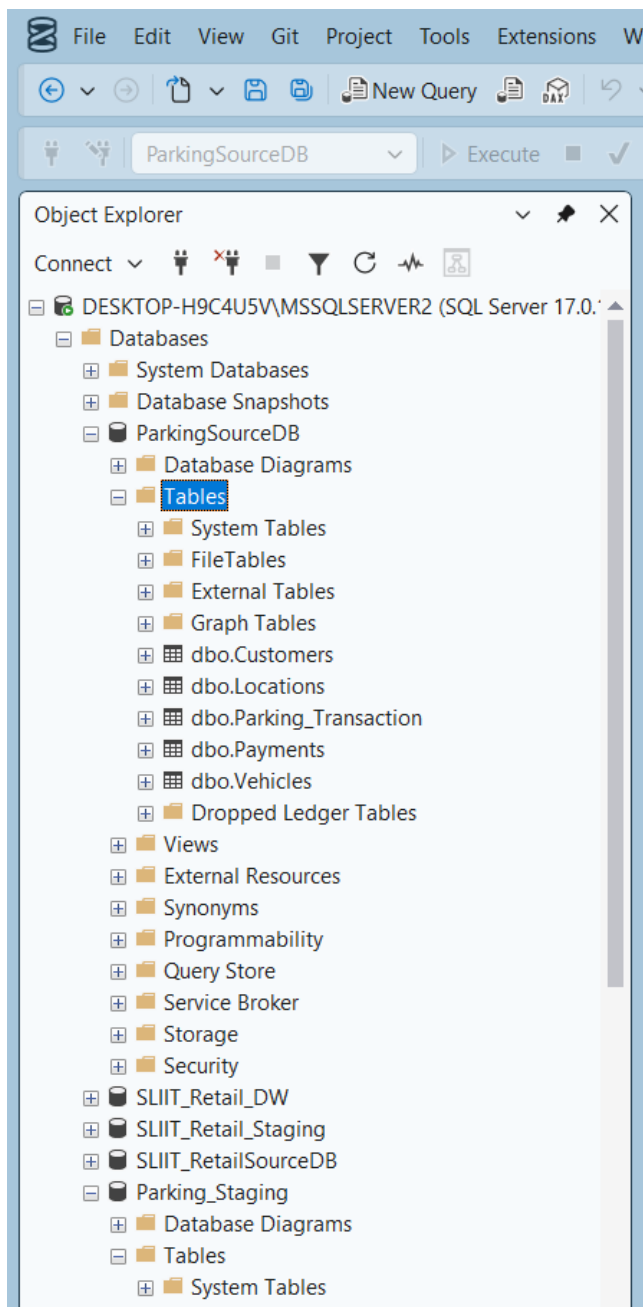
1. Import wizard setup for each table



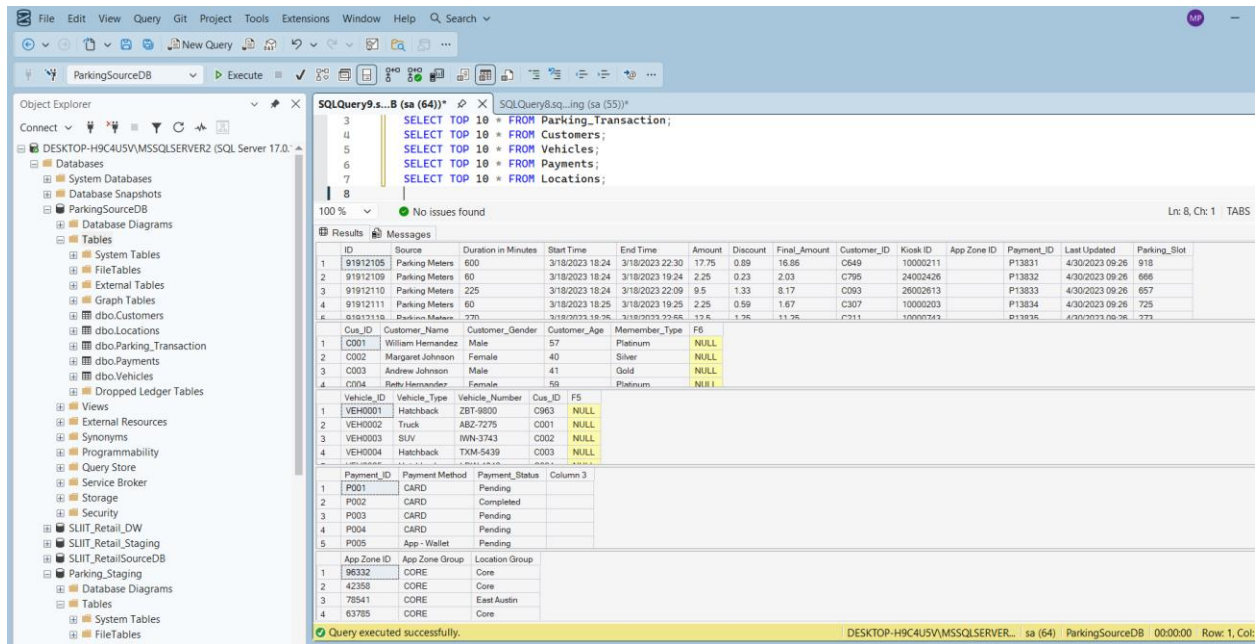




2. Tables created under ParkingSourceDB in Object Explorer



3. Query result for data source files

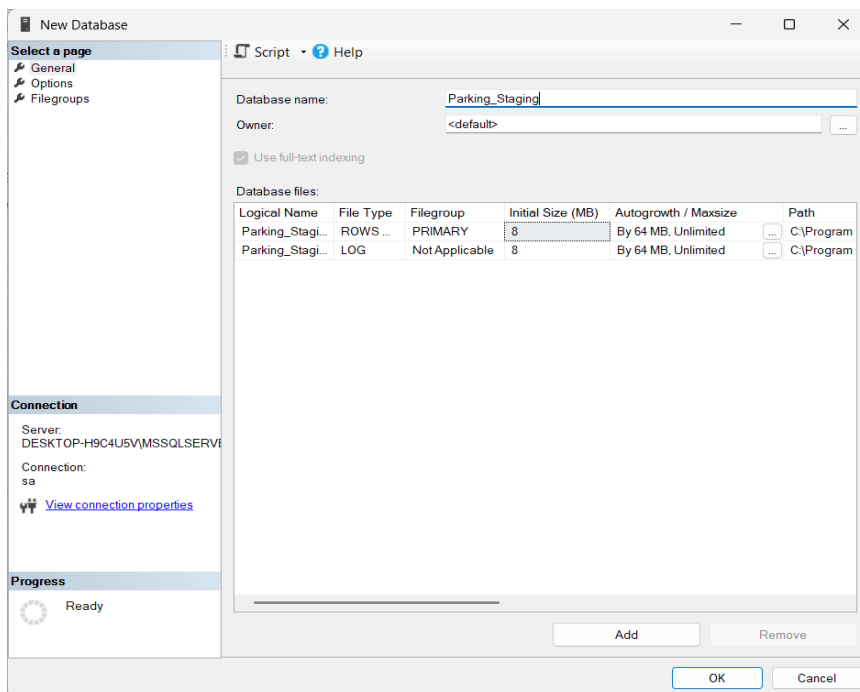


The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the 'ParkingSourceDB' database selected. The right pane shows the query results for the query 'SELECT TOP 10 * FROM Parking_Transaction;'. The results are displayed in a table with columns: ID, Source, Duration in Minutes, Start Time, End Time, Amount, Discount, Final_Amount, Customer_ID, Kiosk ID, App Zone ID, Payment_ID, Last Updated, and Parking_Slot. The status bar at the bottom indicates 'Query executed successfully.'

ID	Source	Duration in Minutes	Start Time	End Time	Amount	Discount	Final_Amount	Customer_ID	Kiosk ID	App Zone ID	Payment_ID	Last Updated	Parking_Slot
1	91912105	Parking Meters	60	3/18/2023 18:24	3/18/2023 22:30	17.75	0.89	16.86	C649	10000211	P13831	4/30/2023 09:26	918
2	91912109	Parking Meters	60	3/18/2023 18:24	3/18/2023 22:09	2.25	0.23	2.03	C795	24002426	P13832	4/30/2023 09:26	666
3	91912110	Parking Meters	225	3/18/2023 18:24	3/18/2023 22:09	9.5	1.33	8.17	C093	26002613	P13833	4/30/2023 09:26	657
4	91912111	Parking Meters	60	3/18/2023 18:25	3/18/2023 22:09	2.25	0.59	1.67	C307	10000203	P13834	4/30/2023 09:26	725
5	91912112	Parking Meters	270	3/18/2023 18:25	3/18/2023 22:09	13.5	1.35	11.95	C311	10000203	P13835	4/30/2023 09:26	773

4. Staging Database Creation

A staging database named Parking_Staging was created in SQL Server. The staging area acts as an intermediate buffer between the source data and the final data warehouse. In the staging area, data is cleaned, validated, and transformed before loading into the data warehouse dimensions and fact tables.



The screenshot shows the 'New Database' dialog box. The 'Database name' field is set to 'Parking_Staging'. The 'Owner' is set to '<default>'. The 'Use full-text indexing' checkbox is checked. The 'Database files' section shows the following configuration:

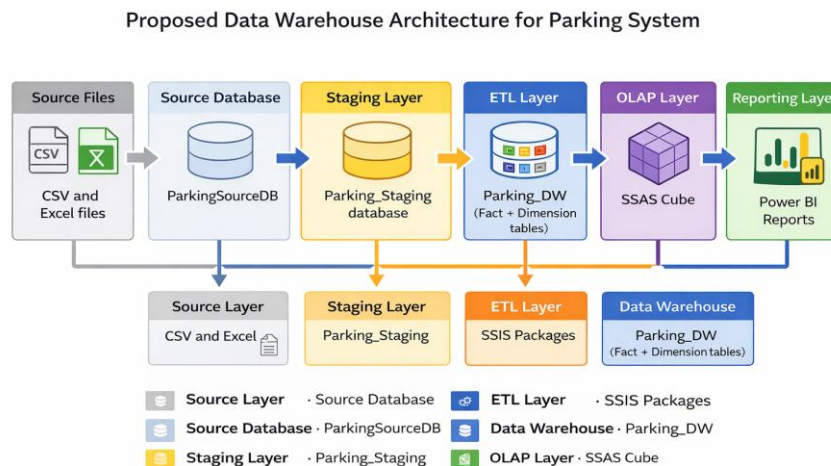
Logical Name	File Type	Filegroup	Initial Size (MB)	Autogrowth / Maxsize	Path
Parking_Stagi...	ROWS ...	PRIMARY	8	By 64 MB, Unlimited	C:\Program
Parking_Stagi...	LOG	Not Applicable	8	By 64 MB, Unlimited	C:\Program

The 'Connection' section shows the server 'DESKTOP-H9C4U5V\MSSQLSERVER' and the connection 'sa'. The 'Progress' section shows 'Ready'. The 'Add' and 'Remove' buttons are visible at the bottom.

Task 3: Solution Architecture

3.1 High-Level Architecture Diagram

The following diagram illustrates the high-level Data Warehouse and Business Intelligence (DW & BI) solution architecture for the Smart Parking Management System:



As illustrated in the figure, the source system (ParkingSourceDB) contains the initially loaded CSV and Excel files, including *Parking_Transaction.csv*, *Customers.xlsx*, *Vehicles.xlsx*, *Payments.csv*, and *Locations.xlsx*. These source files were extracted and transferred into a staging area (Parking_Staging) during the ETL process. The staging database acts as an intermediate storage location where the raw data is cleansed, transformed, and standardized before being loaded into the final data warehouse (Parking_DW). The data warehouse stores the processed data in structured dimension and fact tables to support analysis and reporting.

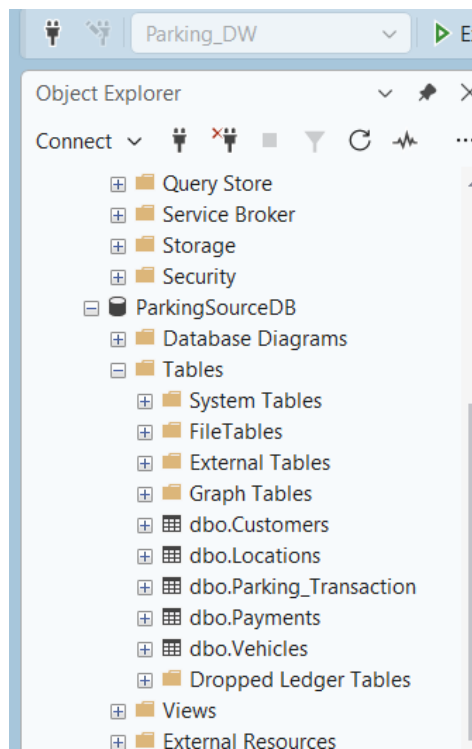
3.2 Architecture Component Descriptions

The DW & BI solution starts with raw data stored in CSV and Excel files. These files are loaded into a source database (ParkingSourceDB). The data is then moved to a staging database (Parking_Staging) where cleaning and transformations are done. After that, the processed data is loaded into the data warehouse (Parking_DW) designed using a star schema. An SSAS cube is built for analysis, and Power BI is used to create reports and dashboards.

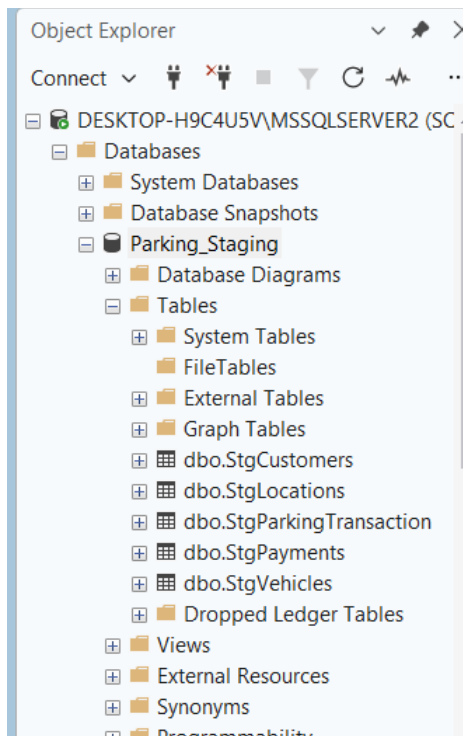
Summarized Descriptions of Each Component

Component	Description
CSV & Excel Files	Raw data files such as Parking_Transaction.csv, Payments.csv, Customers.xlsx, Vehicles.xlsx, and Locations.xlsx.
ParkingSourceDB	Database where raw data is initially loaded and stored.
Parking_Staging	Temporary database used for cleaning and transforming data.
SSIS ETL Packages	Used to extract, transform, and load data into the warehouse.
Parking_DW	Main data warehouse with dimension and fact tables.
SSAS Cube	Used for fast analysis with measures and hierarchies.
Power BI	Used to create dashboards and reports for users.

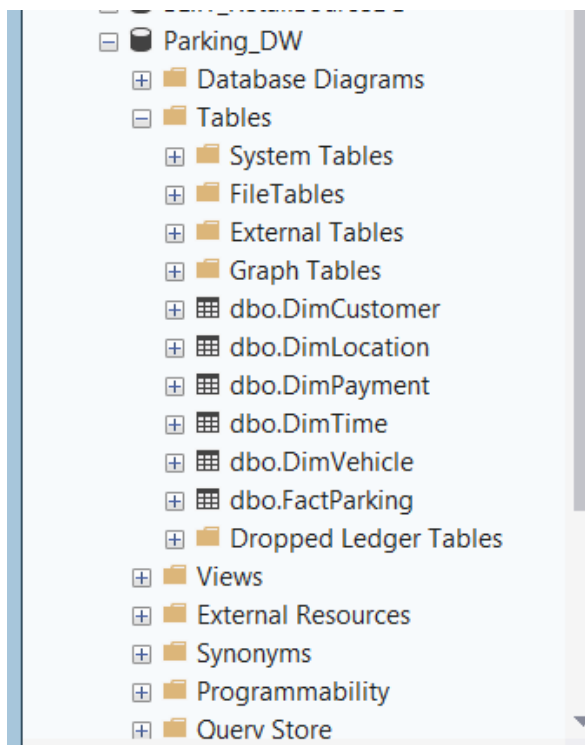
Source Database – ParkingSourceDB



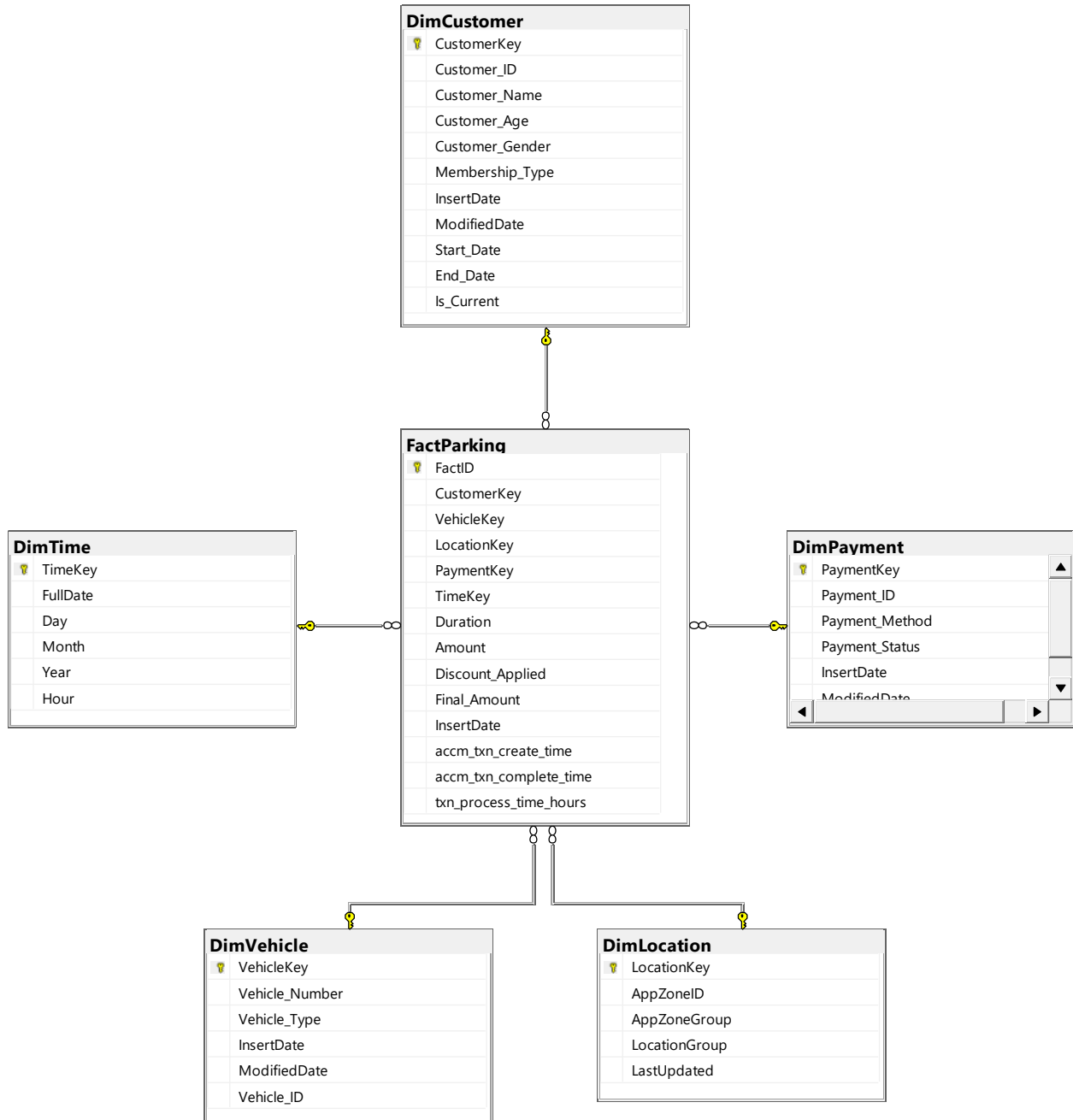
Staging Database – Parking_Staging



Data Warehouse – Parking_DW



Task 4: Data Warehouse Design & Development



- In this data warehouse design, a **Star Schema** is used where a central fact table (**FactParking**) is connected to multiple dimension tables.
- The **primary key (surrogate key)** appears on the dimension table side (e.g., *CustomerKey*, *VehicleKey*), while the **foreign keys** are stored in the fact table (**FactParking**), forming one-to-many relationships.
- The **FactParking** table stores transactional data related to parking activities, including duration, amount, discount applied, and final amount.
- The fact table is connected to the following dimensions:
 - **DimCustomer** (customer details and membership)
 - **DimVehicle** (vehicle information)
 - **DimLocation** (parking zones and groups)
 - **DimPayment** (payment method and status)
 - **DimTime** (date and time hierarchy)
- Each dimension provides descriptive attributes that support filtering, grouping, and analysis of parking transactions.

The grain of the FactParking table is defined as one record per parking transaction per customer per vehicle per time instance.

Measures: Amount, Discount, Final_Amount, Duration

Foreign Keys: CustomerKey, VehicleKey, LocationKey, PaymentKey, TimeKey

Slowly Changing Dimension (SCD)

In this design, **DimCustomer** is implemented as a **Slowly Changing Dimension (Type 2)** to track historical changes in customer membership information.

Assumption

DimCustomer is selected as the Slowly Changing Dimension because customer attributes such as **Membership_Type** can change over time (e.g., Silver → Gold → Platinum).

By implementing SCD Type 2:

- Old records are expired using **End_Date**
- New records are inserted with updated values
- **Start_Date and End_Date** track the validity period

- **Is_Current** identifies the latest active record

This allows the system to maintain historical customer data and perform accurate time-based analysis.

Task 5: ETL Development

5.1 ETL Process Overview

The ETL pipeline was developed using SQL Server Integration Services (SSIS). Four SSIS packages were created to manage the complete data flow from source to the data warehouse:

SSIS Package	Purpose
Parking_Load_Staging.dtsx	Extract data from all five source files (CSV and Excel) and load into Parking_Staging tables. Staging tables are truncated before each load.
DataProfiling.dtsx	Profile each staging table to detect null values, data type mismatches, and anomalies.
Parking_Load_DW.dtsx	Transform and load all dimension tables (in correct order) and then load the fact table. Includes SCD processing for DimCustomer and Lookup transformations for surrogate key resolution.
UpdateFact.dtsx	Separate package to update FactParkingTransaction with accm_txn_complete_time and compute txn_process_time_hours (Task 6).

5.2 Order of ETL Execution

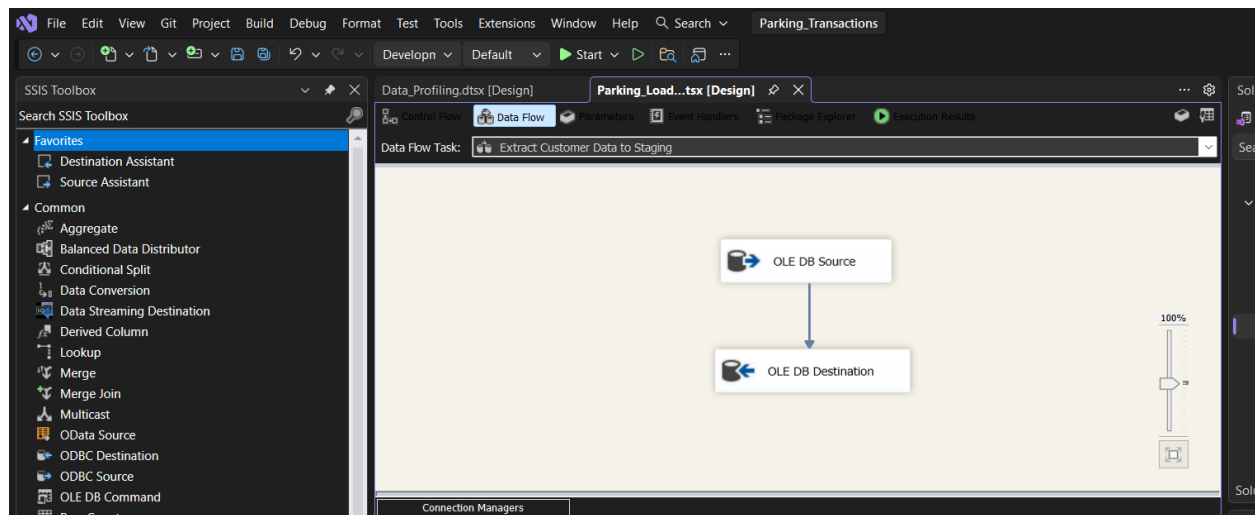
Dimension tables must be loaded before fact tables to ensure foreign key integrity. The correct loading order within Parking_Load_DW.dtsx is:

1. DimDate (pre-populated via SQL script, not SSIS)
2. DimLocation (no dependencies)
3. DimPayment (no dependencies)
4. DimCustomer (SCD Type 2 – processed before vehicles and transactions)
5. DimVehicle (depends on customer natural key)
6. FactParking (all dimension SKs must be available via Lookup)

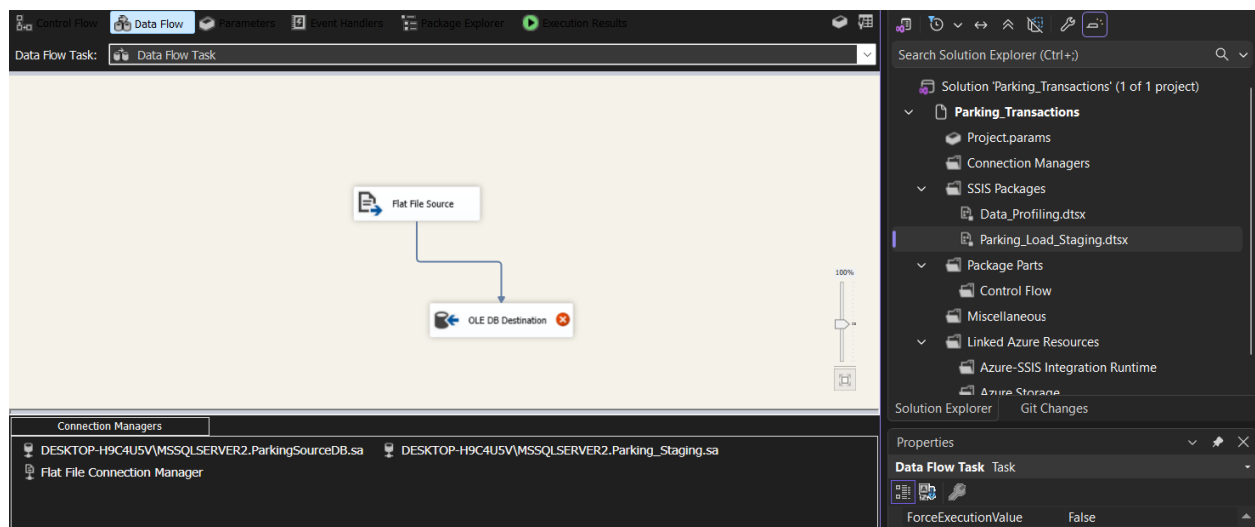
5.3 Extraction – Loading the Staging Tables

There were two types of files in the dataset: CSV files and Excel files. The dataflow tasks were designed accordingly to handle these different file types, as shown in the attached screenshots. For extracting data from CSV files, a Flat File Source component was used in the data flow. For extracting data from Excel files, an Excel File Source component was used.

1.Data Flow Tasks for Excel Files depicted below,



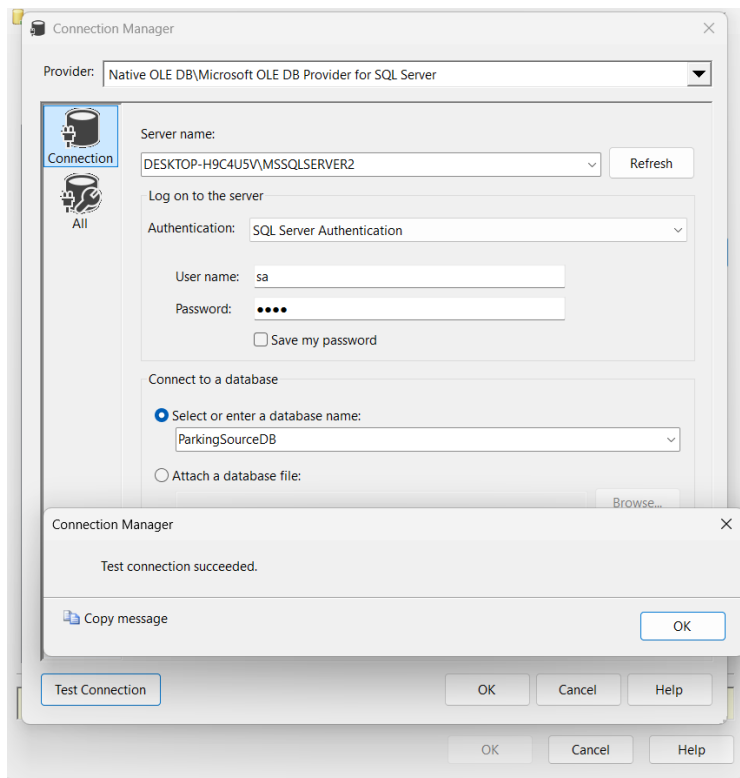
Data Flow Tasks for CSV files are depicted below,



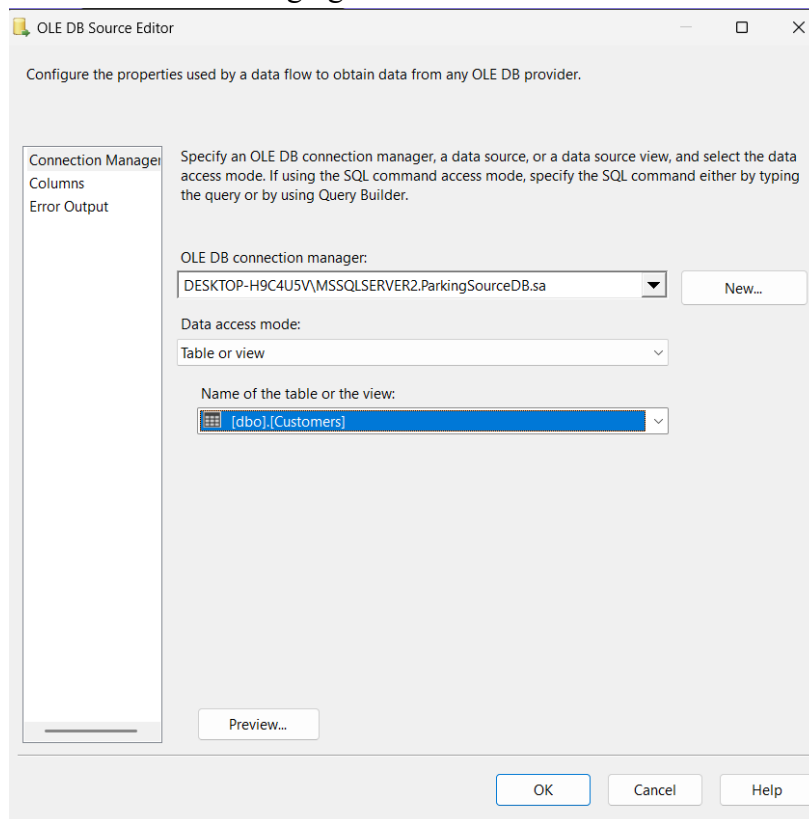
OLE DB Source → is used to read data from a database It would read data from your source table.

OLE DB Destination → is used to write (insert) data into a SQL table (like a staging or warehouse table). It would write the data into your staging table.

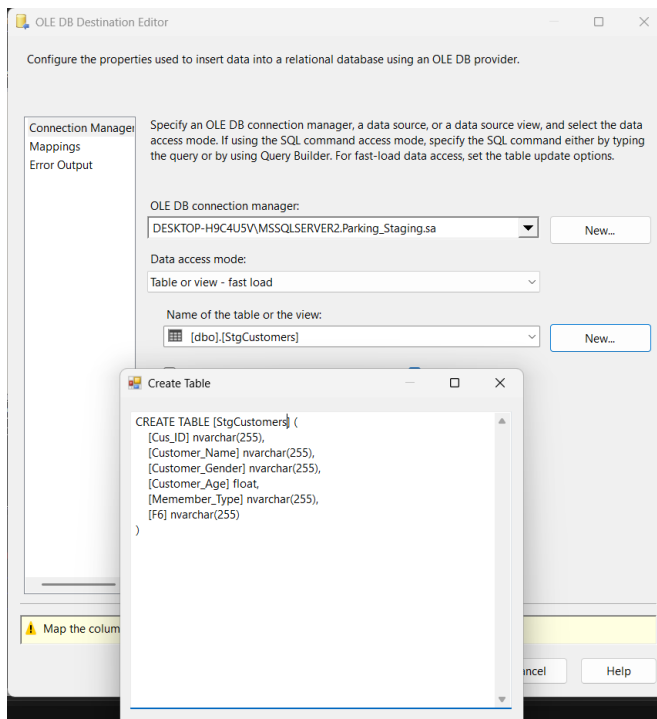
2.Test Connection in OLE DB Source



2. First Customer Staging Table

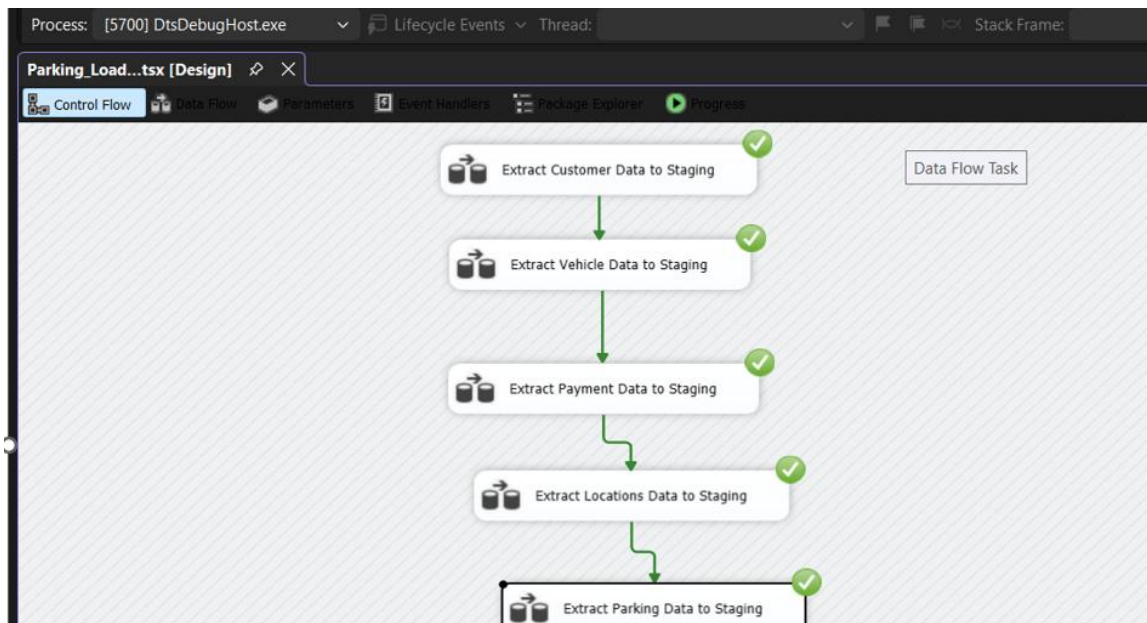


3. Set StgCustomers as the OLE destination

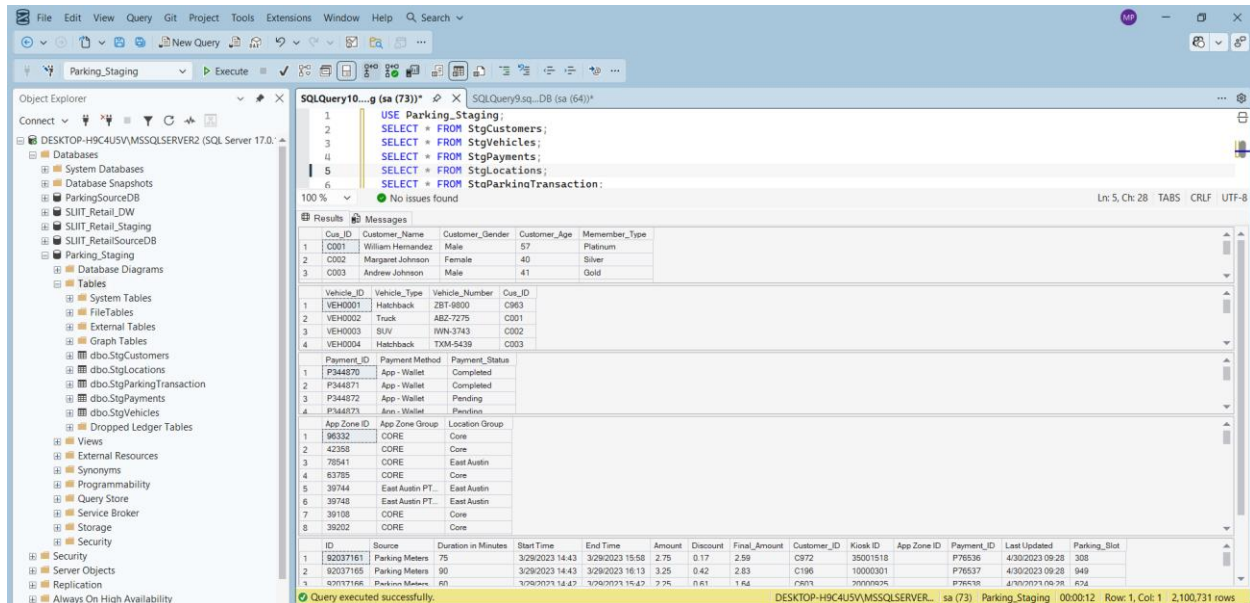


4. The same steps were repeated for every staging table and for each a data flow was created

5. Final Output after Extracting



6. Final Staging Tables after Extracting



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including tables like StgCustomers, StgVehicles, StgLocations, StgPayments, and StgParkingTransaction. The SQL Query window on the right shows a query that selects data from these tables. The Results pane displays the data for each table.

Cus_ID	Customer_Name	Customer_Gender	Customer_Age	Member_Type	
1	C001	William Hernandez	Male	57	Platinum
2	C002	Margaret Johnson	Female	40	Silver
3	C003	Andrew Johnson	Male	41	Gold

Vehicle_ID	Vehicle_Type	Vehicle_Number	Cus_ID	
1	VEH0001	Hatchback	287-8800	C003
2	VEH0002	Truck	ABZ-7275	C001
3	VEH0003	SUV	IWN-3743	C002
4	VEH0004	Hatchback	TXM-5439	C003

Payment_ID	Payment_Method	Payment_Status	
1	P344870	App - Wallet	Completed
2	P344871	App - Wallet	Completed
3	P344872	App - Wallet	Pending
4	P344873	App - Wallet	Pending

App_Zone_ID	App_Zone_Group	Location_Group	
1	96332	CORE	Core
2	42358	CORE	Core
3	78541	CORE	East Austin
4	63785	CORE	Core
5	39744	East Austin PT	East Austin
6	39748	East Austin PT	East Austin
7	39108	CORE	Core
8	39202	CORE	Core

ID	Source	Duration in Minutes	Start Time	End Time	Amount	Discount	Final_Amount	Customer_ID	Kiosk ID	App_Zone ID	Payment_ID	Last Updated	Parking_Slot
1	92037161	Parking Meters	75	3/29/2023 14:43	3/29/2023 15:58	2.75	0.17	2.58	C972	35001518	P76536	4/30/2023 09:28	308
2	92037165	Parking Meters	90	3/29/2023 14:43	3/29/2023 16:13	3.25	0.42	2.83	C196	10000301	P76537	4/30/2023 09:28	949
3	92017168	Parking Meters	60	3/29/2023 14:47	3/29/2023 15:47	2.25	0.61	1.64	C901	20000625	P76538	4/30/2023 09:28	874

7. Truncate Table

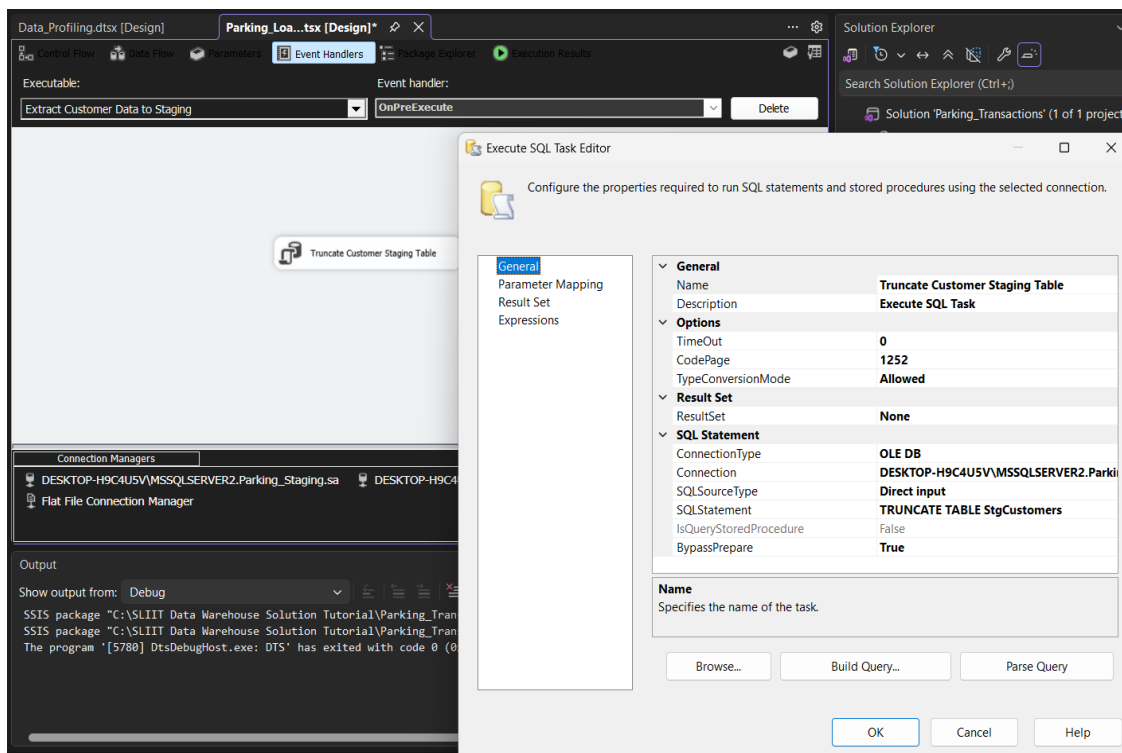
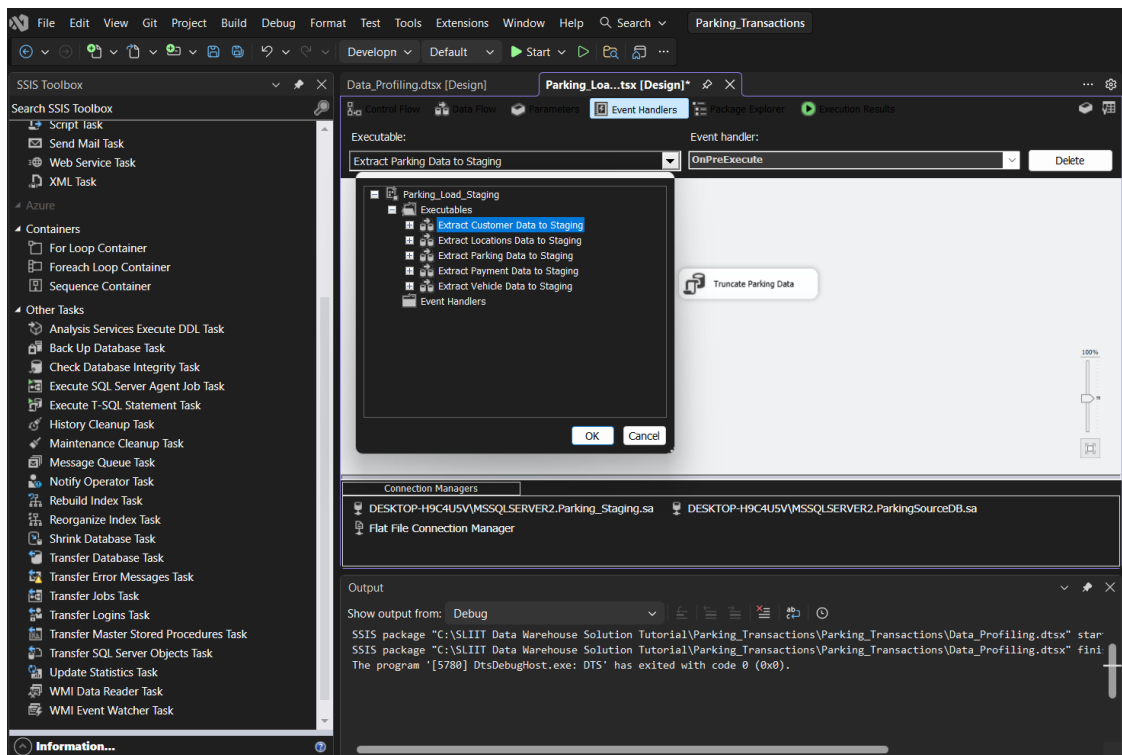
In this project, the TRUNCATE TABLE command was used during the ETL process to clear the staging tables in the **Parking_Staging** database before loading fresh data.

This ensures that each time the ETL package runs, the staging tables start empty, preventing old or duplicate data from remaining in the staging area.

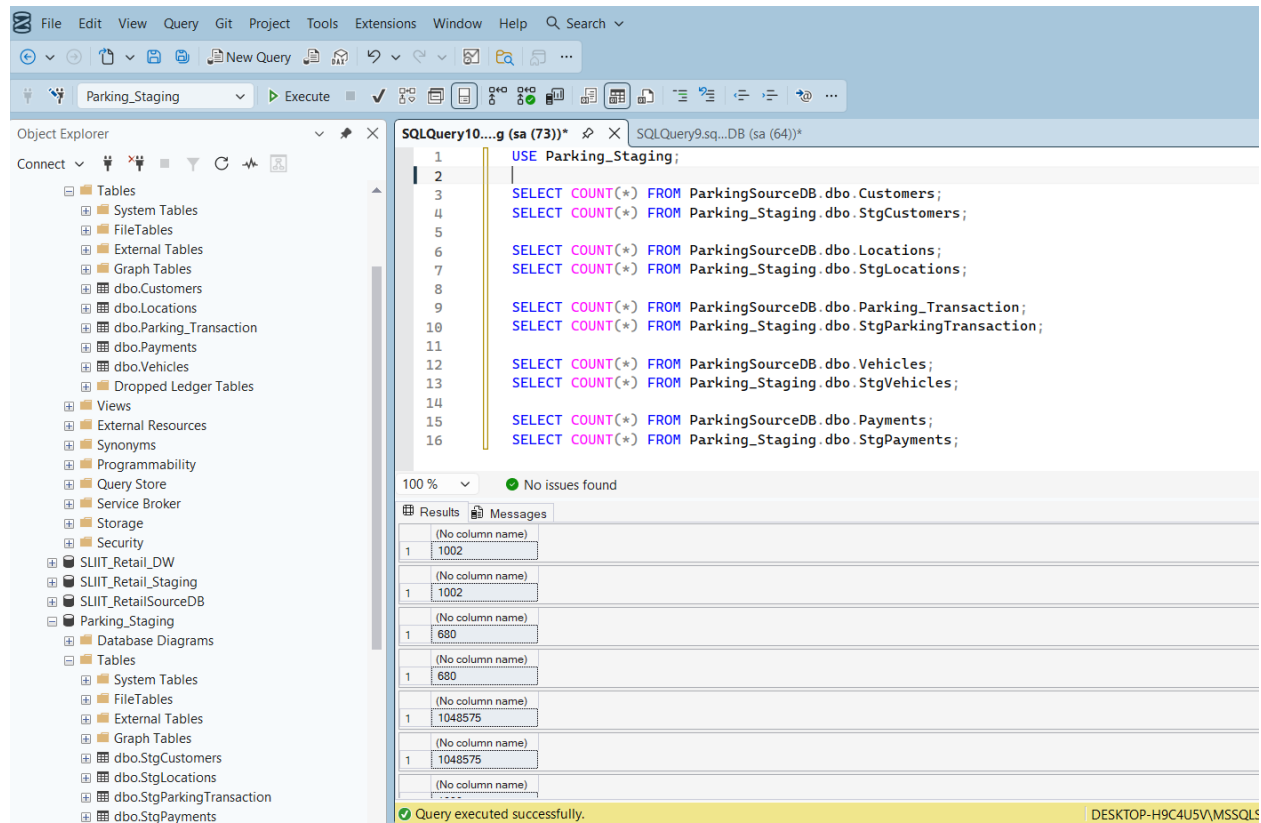
By truncating the staging tables before each data load, the extraction and transformation processes can proceed cleanly, ensuring data consistency, accuracy, and avoiding duplication in the final data warehouse (**Parking_DW**).

The TRUNCATE TABLE operation was applied to all staging tables, including customer, vehicle, location, payment, and parking transaction staging tables

Below screenshot shows the how truncate command use for Customer table for all the other tables also this process continued.....



Confirming all source table data are loaded into staging tables,

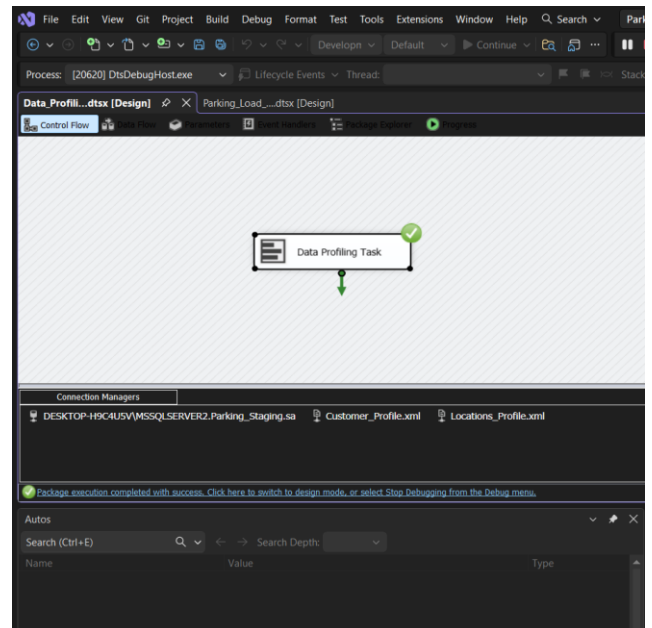
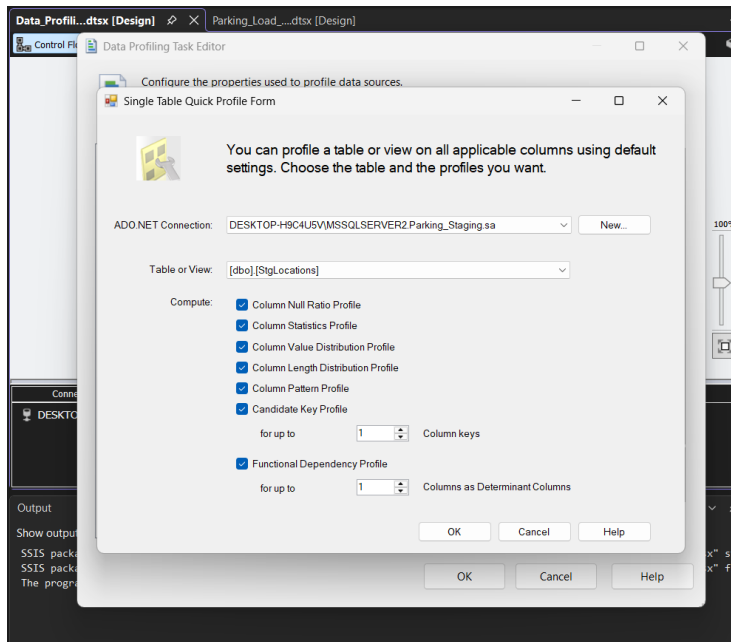
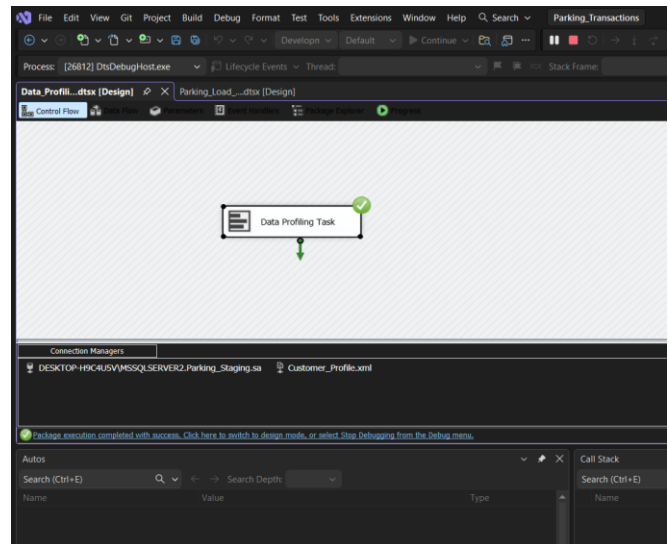
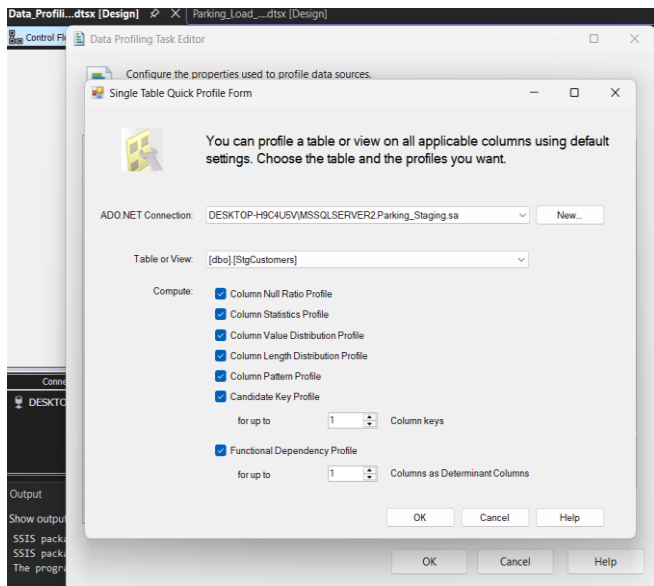


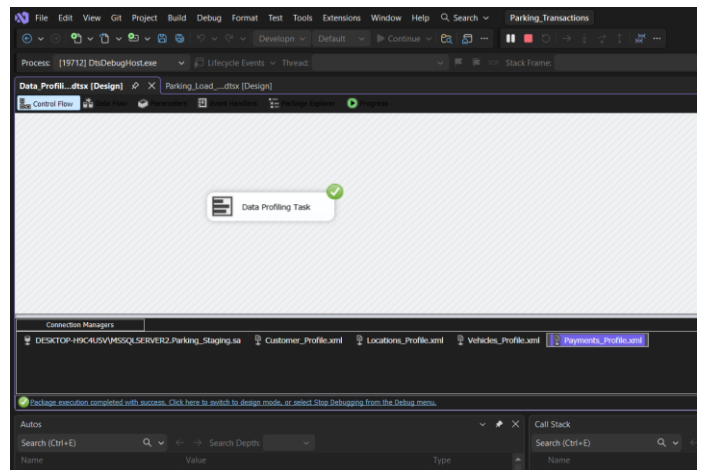
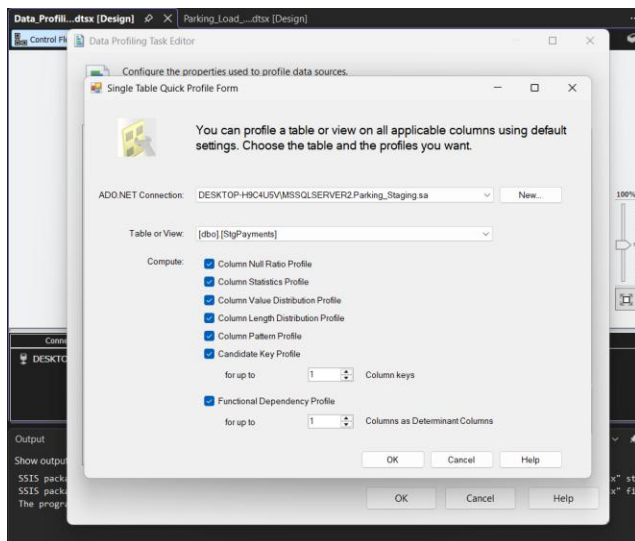
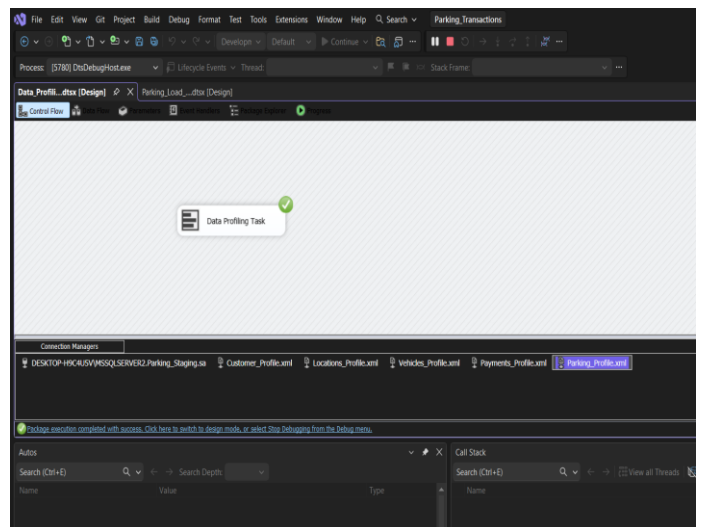
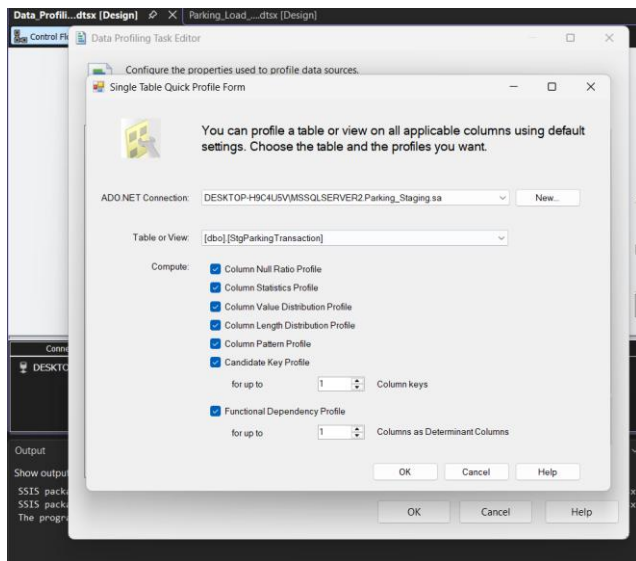
Data validation was performed by comparing row counts between source and staging tables to ensure data consistency and completeness.

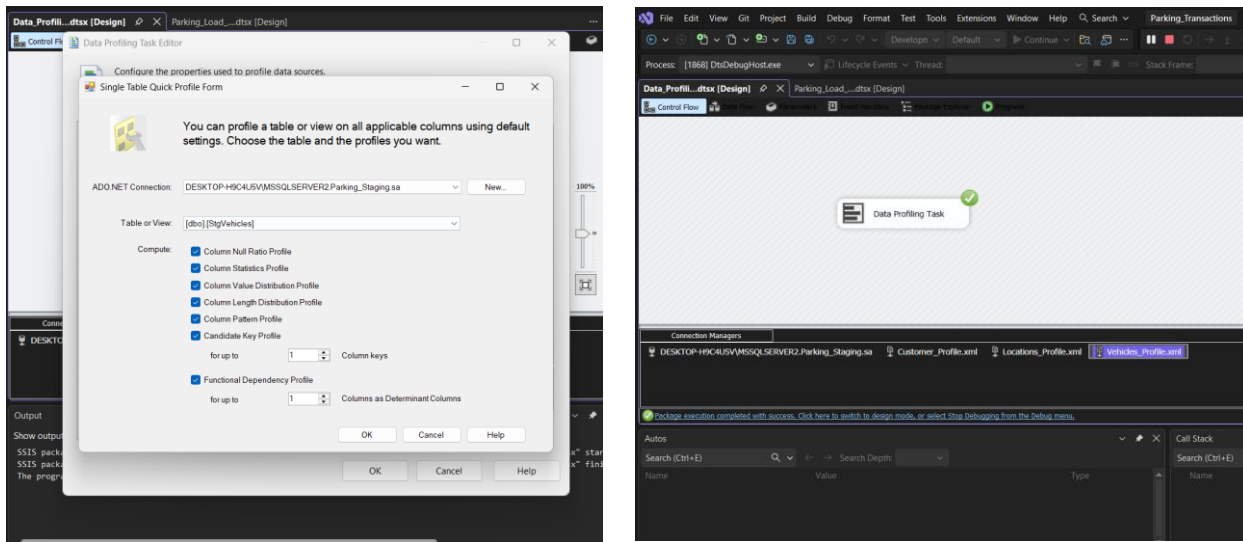
8. Data Profiling

Data profiling was performed using the SSIS Data Profiling Task on each staging table to identify data quality issues such as null values, column statistics, value distribution, and candidate key violations. The profile output was saved as XML files and reviewed using the Data Profile Viewer.

Data profiling identified null values, column distributions, and potential data quality issues. These insights were used to improve data consistency before loading into the data warehouse.







5.4 Error Handling in ETL

Error handling was implemented in the SSIS ETL process to manage data quality issues and ensure reliable data loading. Error outputs were configured in data flow components to redirect invalid or failed records into separate error tables for further analysis.

Additionally, logging mechanisms were used to monitor ETL execution, capture failures, and track the success or failure of data loads. This ensures that any data issues can be identified and corrected without interrupting the entire ETL pipeline.

5.5 Transformation and Loading

Creating the Dimensions and facts tables in SSMS

```
SQLQuery69....W (sa (74))* X
1 CREATE TABLE Dim_Location (
2     LocationKey INT IDENTITY(1,1) PRIMARY KEY,
3     AppZoneID INT,
4     AppZoneGroup VARCHAR(50),
5     LocationGroup VARCHAR(50),
6     LastUpdated DATETIME DEFAULT GETDATE()
7 );
8
9 CREATE TABLE Dim_Customer (
10     CustomerKey INT IDENTITY(1,1) PRIMARY KEY,
11     Customer_ID INT,
12     Customer_Name VARCHAR(100),
13     Customer_Age INT,
14     Customer_Gender VARCHAR(10),
15     Membership_Type VARCHAR(20),
16     InsertDate DATETIME DEFAULT GETDATE(),
17     ModifiedDate DATETIME,
18     Start_Date DATETIME DEFAULT GETDATE(),
19     End_Date DATETIME NULL,
20     Is_Current BIT DEFAULT 1
21 );
22 CREATE TABLE Dim_Payment (
23     PaymentKey INT IDENTITY(1,1) PRIMARY KEY,
24     Payment_ID INT,
25     Payment_Method VARCHAR(50),
26     Payment_Status VARCHAR(50),
27     InsertDate DATETIME DEFAULT GETDATE(),
28     ModifiedDate DATETIME
29 );
30 CREATE TABLE Dim_Time (
31     TimeKey INT PRIMARY KEY,
32     FullDate DATE,
33     Day INT,
34     Month INT,
35     Year INT,
36     Hour INT
37 );
38 CREATE TABLE Dim_Vehicle (
39     VehicleKey INT IDENTITY(1,1) PRIMARY KEY,
40     Vehicle_ID INT,
41     Vehicle_Number VARCHAR(20),
42     Vehicle_Type VARCHAR(50),
43     InsertDate DATETIME DEFAULT GETDATE(),
44     ModifiedDate DATETIME
45 );
46
47 CREATE TABLE FactParking (
48     FactID INT IDENTITY(1,1) PRIMARY KEY,
49
50     CustomerKey INT,
51     VehicleKey INT,
52     LocationKey INT,
53     PaymentKey INT,
54     TimeKey INT,
55
56     Duration INT,
57     Amount DECIMAL(10,2),
58     Discount_Applied DECIMAL(10,2),
59     Final_Amount DECIMAL(10,2),
60
61     InsertDate DATETIME DEFAULT GETDATE(),
62
63     accm_txn_create_time DATETIME,
64     accm_txn_complete_time DATETIME,
65     txn_process_time_hours INT,
66
67     FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey),
68     FOREIGN KEY (VehicleKey) REFERENCES DimVehicle(VehicleKey),
69     FOREIGN KEY (LocationKey) REFERENCES DimLocation(LocationKey),
70     FOREIGN KEY (PaymentKey) REFERENCES DimPayment(PaymentKey),
71     FOREIGN KEY (TimeKey) REFERENCES DimTime(TimeKey)
72 );
```

Stored procedures were used to modularize ETL logic, improve performance, and ensure reusability. They also help maintain data integrity and simplify maintenance.

```

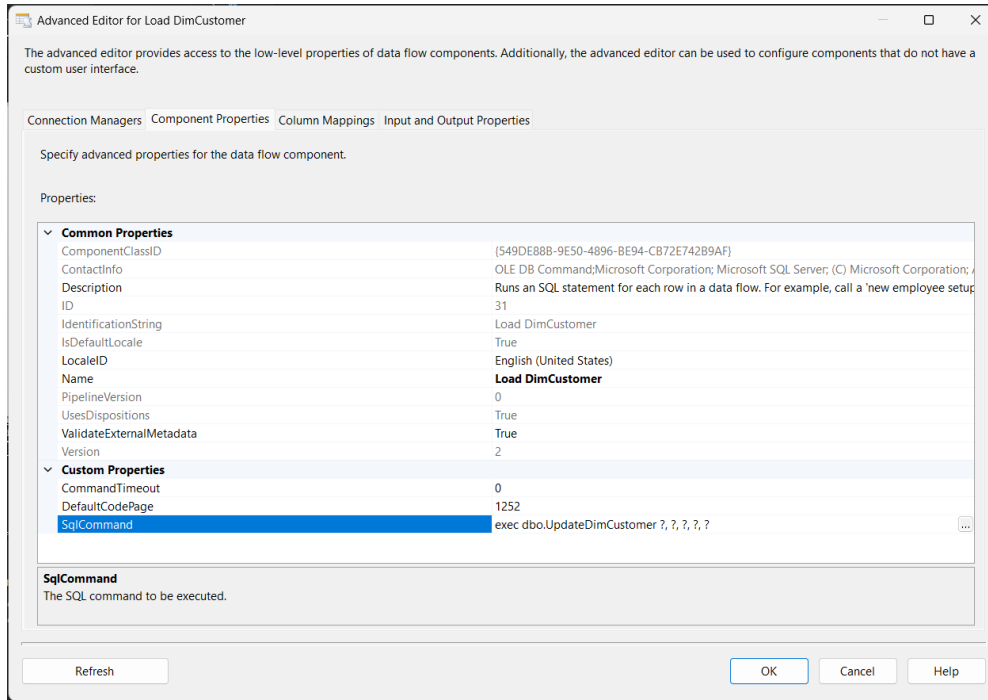
SQLQuery70....W (sa (52))*  SQLQuery69.s...DW (sa (74))*
1      CREATE PROCEDURE sp_UpdateDimCustomer
2          @Customer_ID INT,
3          @Customer_Name VARCHAR(100),
4          @Customer_Age INT,
5          @Customer_Gender VARCHAR(10),
6          @Membership_Type VARCHAR(20)
7      AS
8      BEGIN
9          IF EXISTS (
10             SELECT 1 FROM DimCustomer
11             WHERE Customer_ID = @Customer_ID AND Is_Current = 1
12         )
13         BEGIN
14             -- Expire old record
15             UPDATE DimCustomer
16             SET End_Date = GETDATE(),
17                 Is_Current = 0
18             WHERE Customer_ID = @Customer_ID
19                 AND Is_Current = 1;
20
21             -- Insert new version
22             INSERT INTO DimCustomer (
23                 Customer_ID, Customer_Name, Customer_Age,
24                 Customer_Gender, Membership_Type,
25                 Start_Date, Is_Current
26             )
27             VALUES (
28                 @Customer_ID, @Customer_Name, @Customer_Age,
29                 @Customer_Gender, @Membership_Type,
30                 GETDATE(), 1
31             );
32         END
33     ELSE
34     BEGIN
35         INSERT INTO DimCustomer (
36             Customer_ID, Customer_Name, Customer_Age,
37             Customer_Gender, Membership_Type,
38             Start_Date, Is_Current
39         )
40         VALUES (
41             @Customer_ID, @Customer_Name, @Customer_Age,
42             @Customer_Gender, @Membership_Type,
43             GETDATE(), 1
44         );
45     END
46 END;

47
48 90;
49 CREATE PROCEDURE sp_UpdateDimLocation
50     @AppZoneID INT,
51     @AppZoneGroup VARCHAR(50),
52     @LocationGroup VARCHAR(50)
53 AS
54 BEGIN
55     IF NOT EXISTS (
56         SELECT 1 FROM DimLocation WHERE AppZoneID = @AppZoneID
57     )
58     BEGIN
59         INSERT INTO DimLocation (AppZoneID, AppZoneGroup, LocationGroup)
60         VALUES (@AppZoneID, @AppZoneGroup, @LocationGroup);
61     END
62     ELSE
63     BEGIN
64         UPDATE DimLocation
65         SET AppZoneGroup = @AppZoneGroup,
66             LocationGroup = @LocationGroup,
67             LastUpdated = GETDATE()
68         WHERE AppZoneID = @AppZoneID;
69     END;
70
71
72 CREATE PROCEDURE sp_UpdateDimPayment
73     @Payment_ID INT,
74     @Payment_Method VARCHAR(50),
75     @Payment_Status VARCHAR(50)
76 AS
77 BEGIN
78     IF NOT EXISTS (
79         SELECT 1 FROM DimPayment WHERE Payment_ID = @Payment_ID
80     )
81     BEGIN
82         INSERT INTO DimPayment (Payment_ID, Payment_Method, Payment_Status)
83         VALUES (@Payment_ID, @Payment_Method, @Payment_Status);
84     END
85     ELSE
86     BEGIN
87         UPDATE DimPayment
88         SET Payment_Method = @Payment_Method,
89             Payment_Status = @Payment_Status,
90             ModifiedDate = GETDATE()
91         WHERE Payment_ID = @Payment_ID;
92     END
93 END;

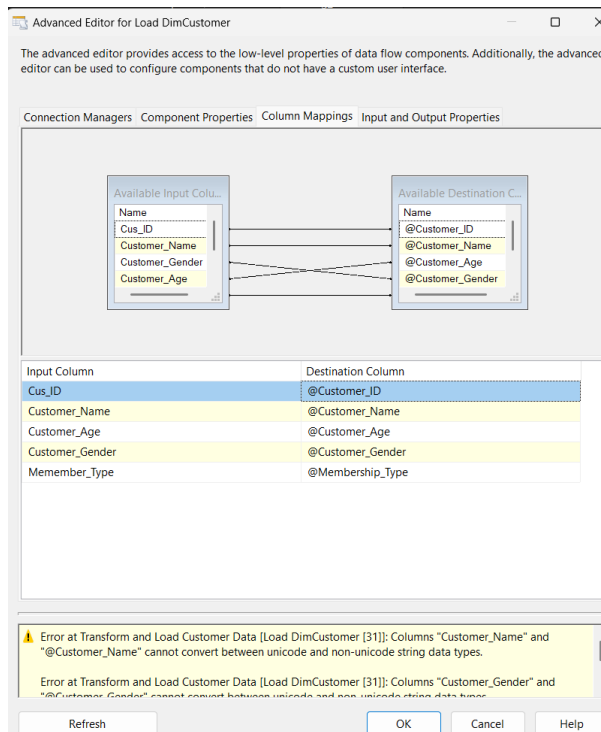
```

Loading DimCustomer

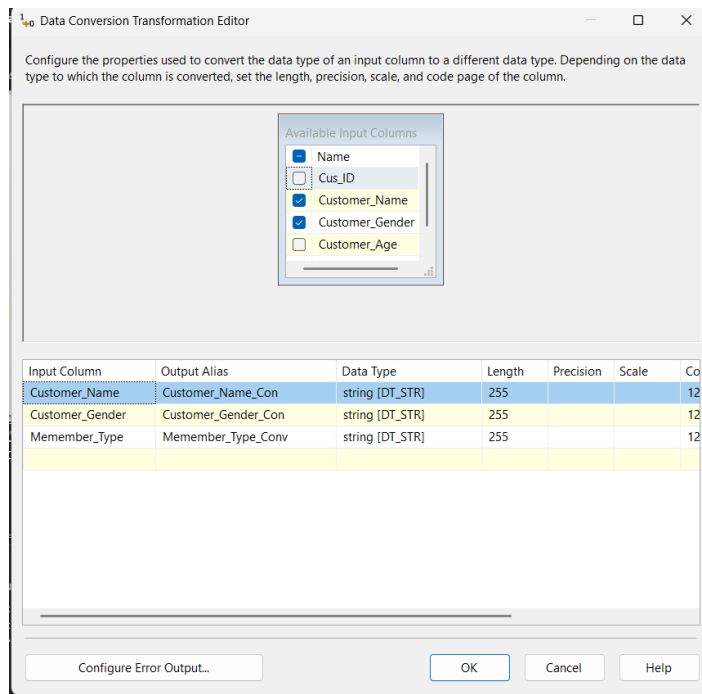
1. Making connection in lookup



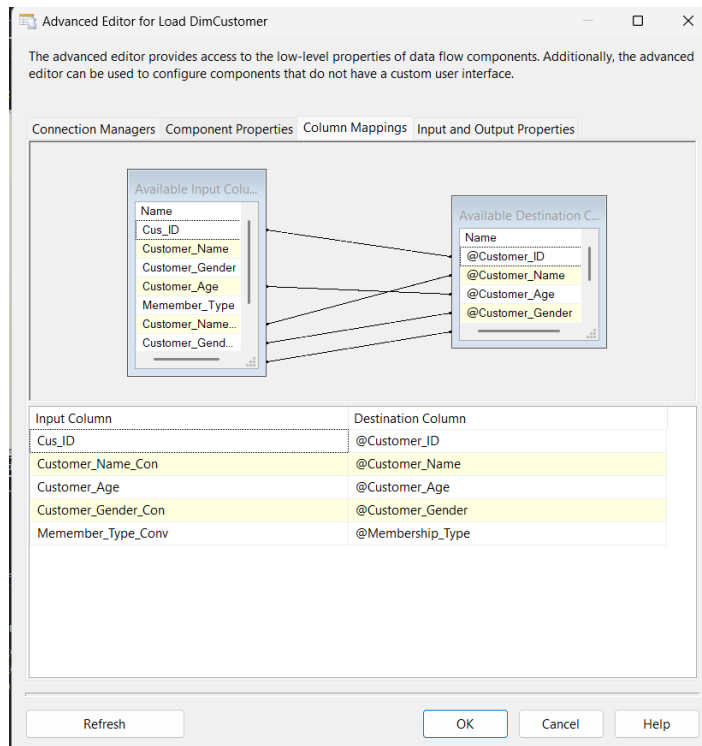
2. Mapping columns in lookup



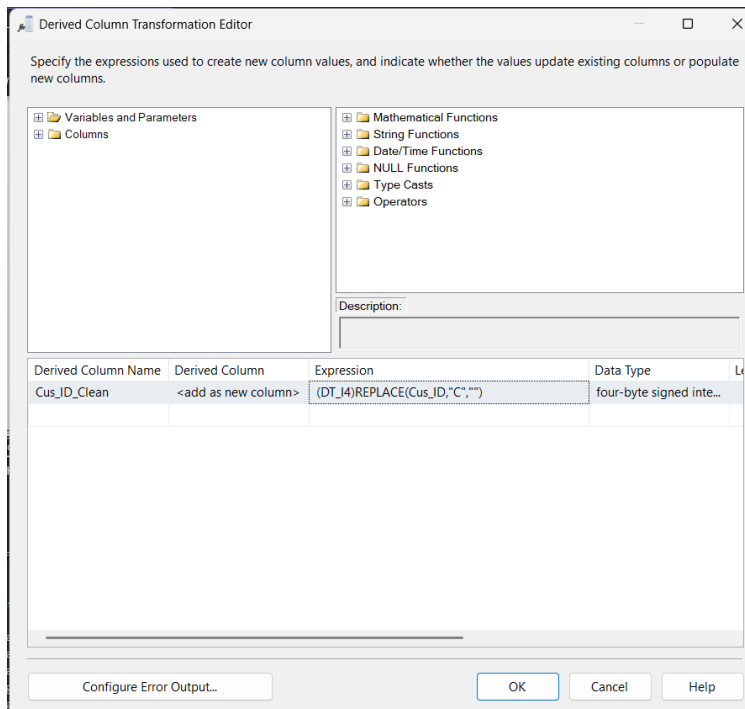
3. Data conversion was done



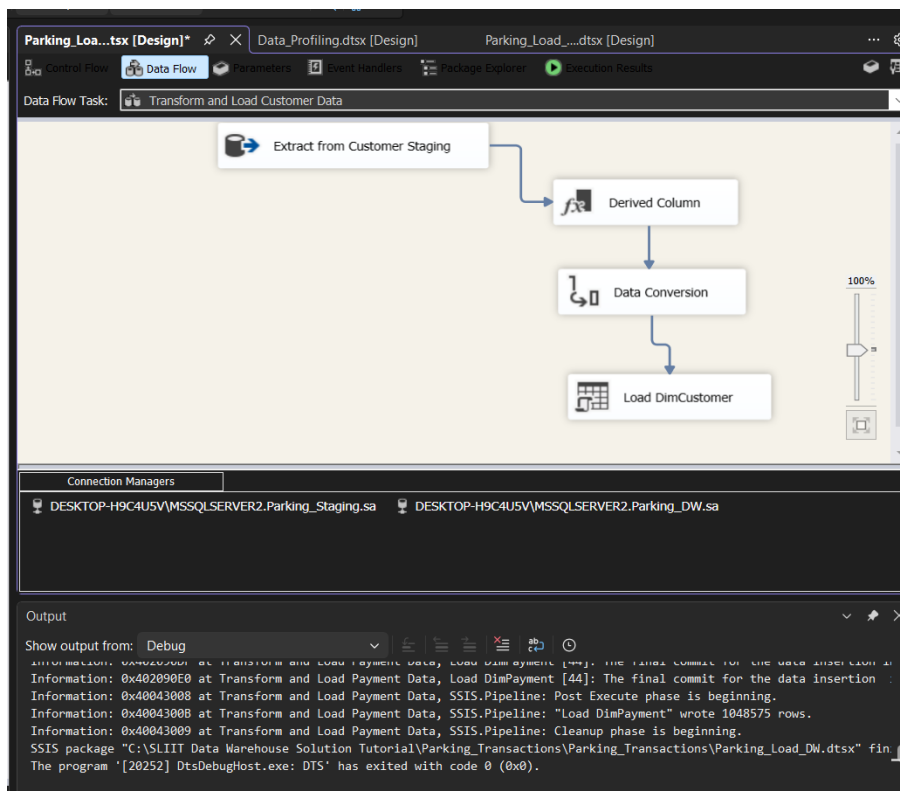
4.



5.A Derived Column was made



6.Full data flow of Customer Staging table

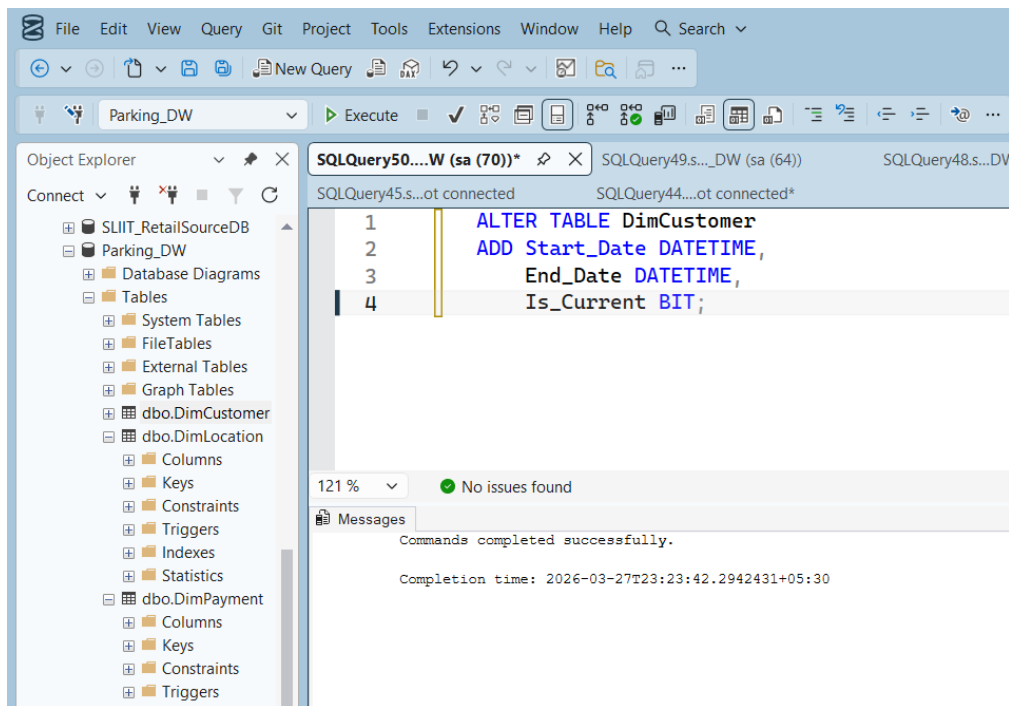


7. Implementing Slowly Changing Dimension

In this project, the **DimCustomer** table is implemented as a **Slowly Changing Dimension Type 2**. This is used to track changes in customer attributes such as **Membership_Type**. When a customer's membership changes (e.g., from Silver to Gold), the existing record is not overwritten. Instead, the old record is expired and a new record is inserted with the updated value.

To support this, additional columns are used:

- **Start_Date** – indicates when the record became active
- **End_Date** – indicates when the record expired
- **Is_Current** – identifies whether the record is the latest version



Slowly Changing Dimension Wizard

Select a Dimension Table and Keys

Select a dimension table to load and map columns in the transformation input to columns in the dimension table.

Connection manager:
 DESKTOP-H9C4U5V\MSSQLSERVER2.Parking_DW.sa New...

Table or view:
 [dbo].[DimCustomer] ▼

Input Columns	Dimension Columns	Key Type
Customer_Age_Conv	Customer_Age	Not a key column
Customer_Gender	Customer_Gender	Not a key column
Cus_ID_Clean_Conv	Customer_ID	Business key
Customer_Name	Customer_Name	Not a key column
	End_Date	
	InsertDate	▼
	Is_Current	

Slowly Changing Dimension Wizard

Select a Dimension Table and Keys

Select a dimension table to load and map columns in the transformation input to columns in the dimension table.

Connection manager:
 DESKTOP-H9C4U5V\MSSQLSERVER2.Parking_DW.sa New...

Table or view:
 [dbo].[DimCustomer] ▼

Input Columns	Dimension Columns	Key Type
Customer_Name	Customer_Name	Not a key column
	End_Date	
	InsertDate	
	Is_Current	
Member_Type	Membership_Type	Not a key column
	ModifiedDate	
	Start_Date	

Slowly Changing Dimension Wizard

Slowly Changing Dimension Columns

Manage the changes to column data in your slowly changing dimensions by setting the change type for dimension columns.

Fixed Attribute

Select this type when the value in a column should not change. Changes are treated as errors.

Changing Attribute

Select this type when changed values should overwrite existing values. This is a Type 1 change.

Historical Attribute

Select this type when changes in column values are saved in new records. Previous values are saved in records marked as outdated. This is a Type 2 change.

Select a change type for slowly changing dimension columns:

Dimension Columns	Change Type
Customer_Age	Changing a...
Customer_Gender	Fixed attrib...
Customer_Name	Fixed attrib...
Membership_Type	Historical a...

Help

< Back

Next >

Finish >>|

Slowly Changing Dimension Wizard

Historical Attribute Options

You can record historical attributes using a single column or start and end date columns.

☐ Use a single column to show current and expired records

Column to indicate current record:

Value when current:

Expiration value:

☒ Use start and end dates to identify current and expired records

Start date column:

Start_Date

End date column:

End_Date

Variable to set date values:

System::StartTime

Help

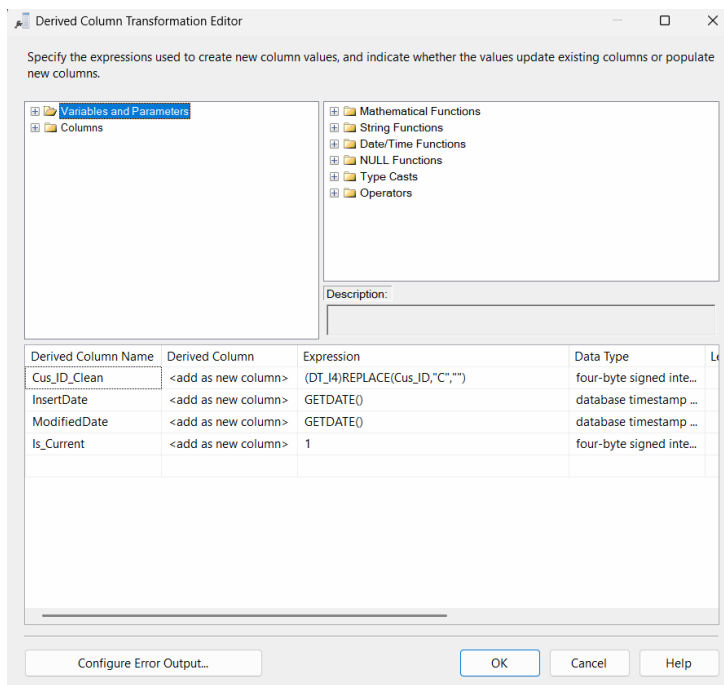
< Back

Next >

Finish >>|

Cancel

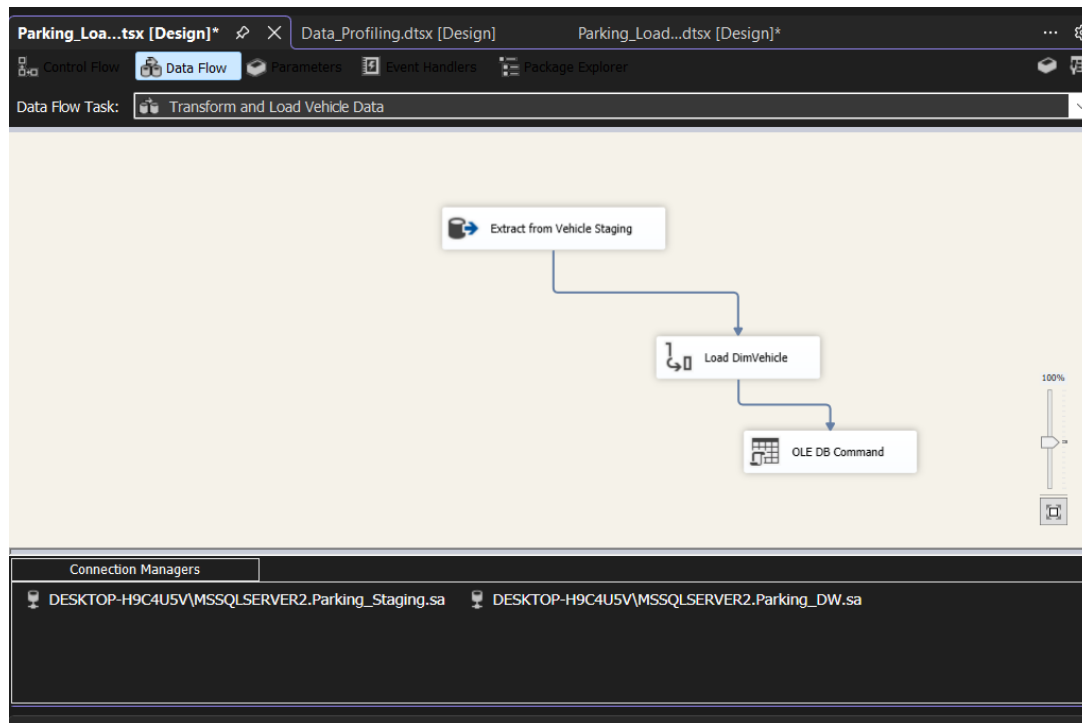
33 | Page



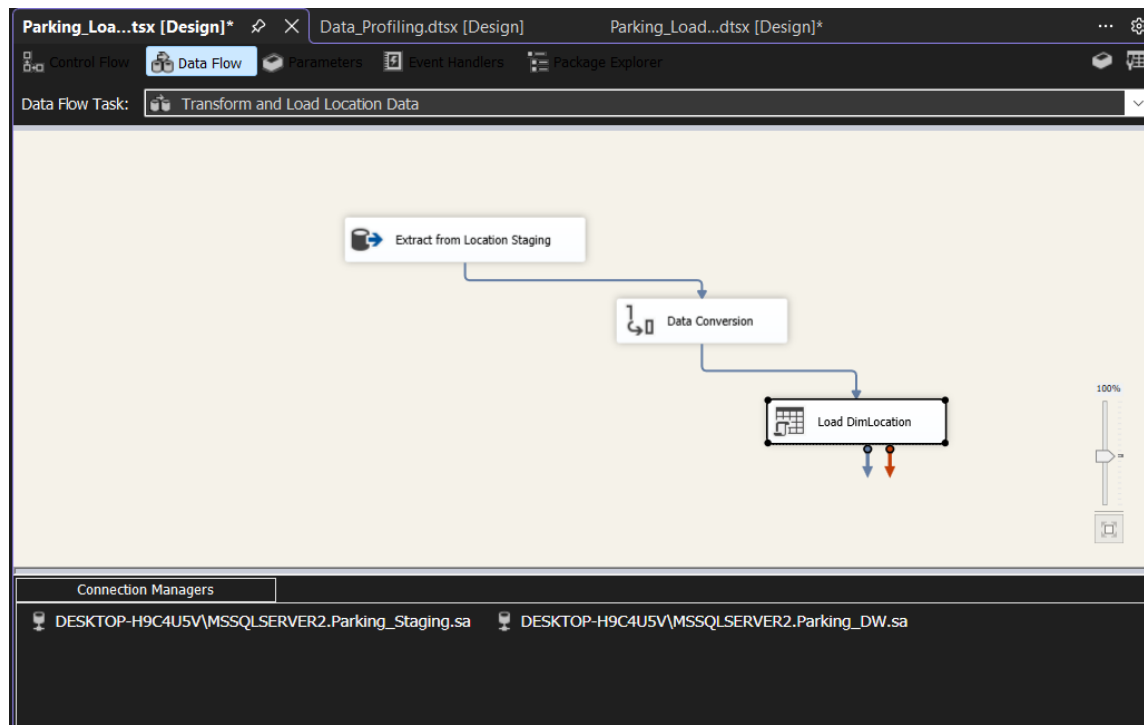
Final SCD Data Flow



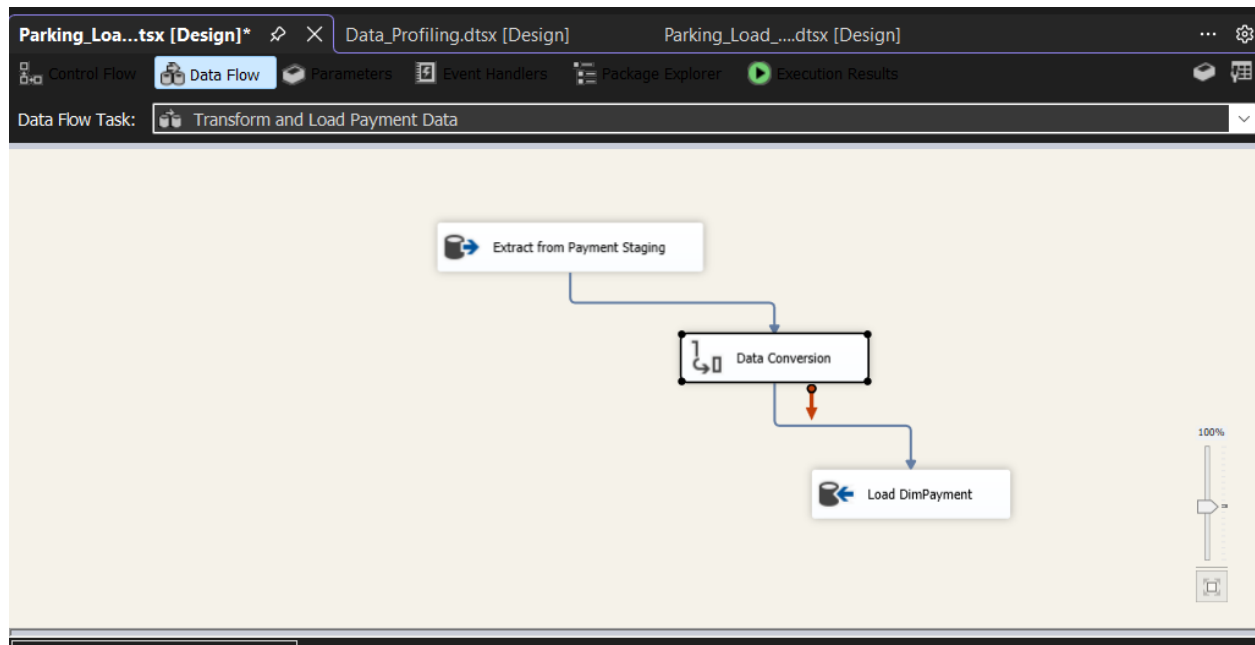
Loading DimVehicle



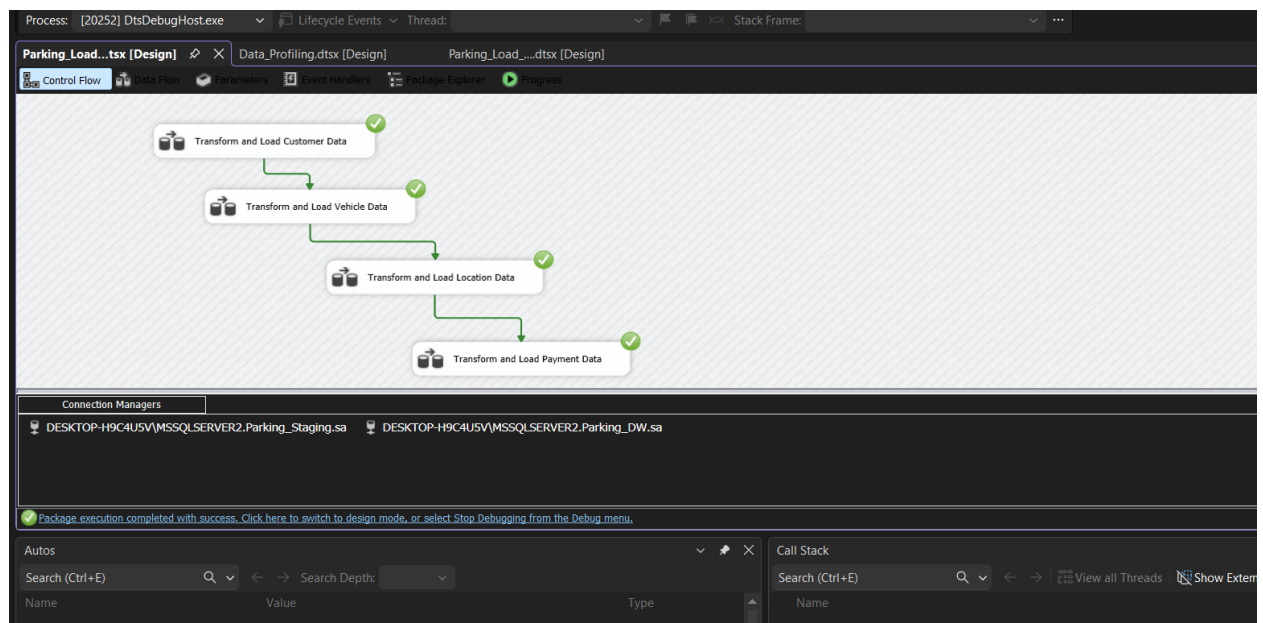
Loading DimLocation



Loading DimPayment

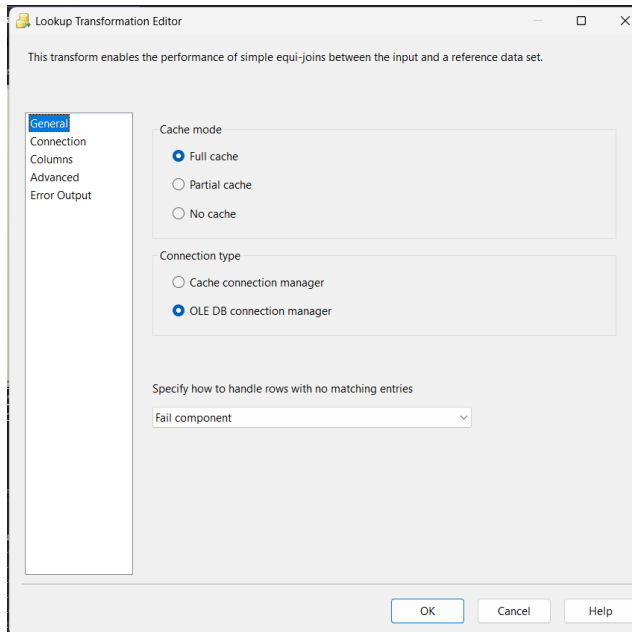


Final all dimension tables workflows

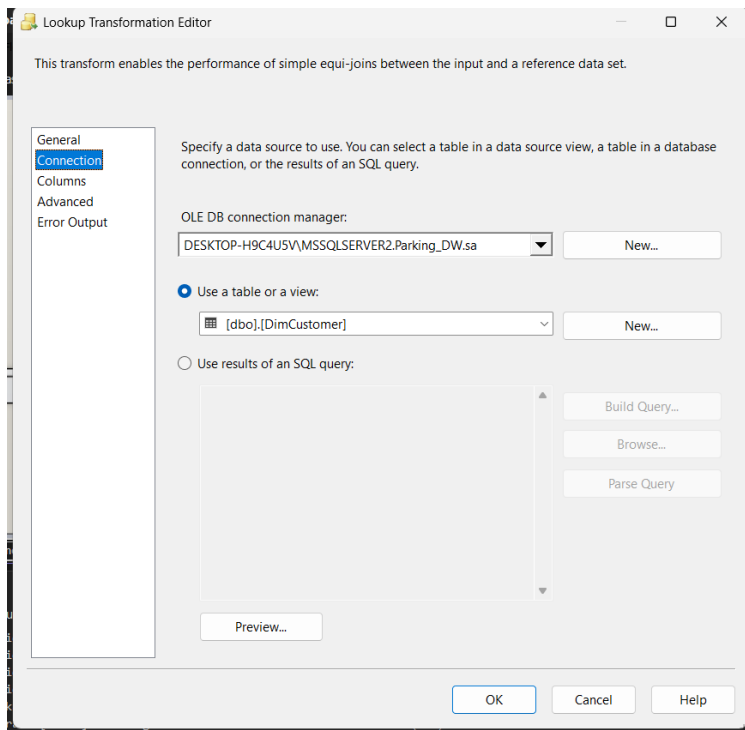


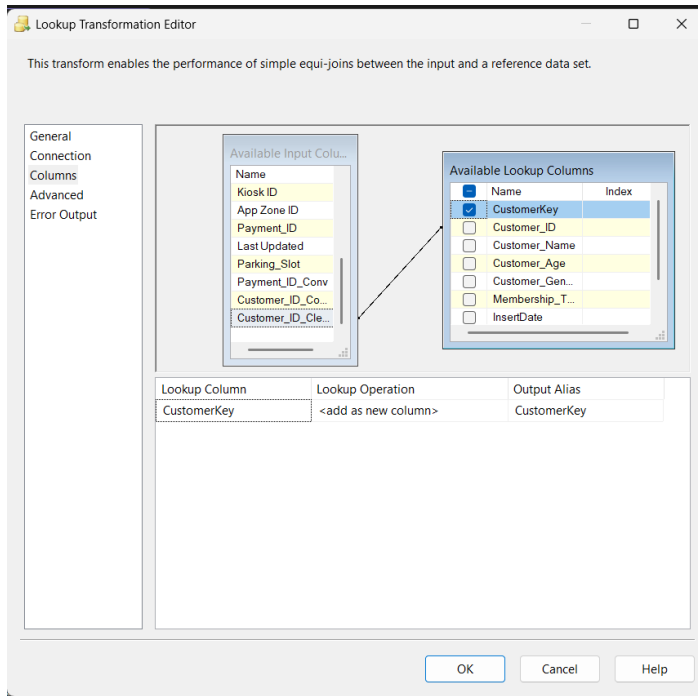
Loading FactParkingTransaction

1. A lookup is made for customer

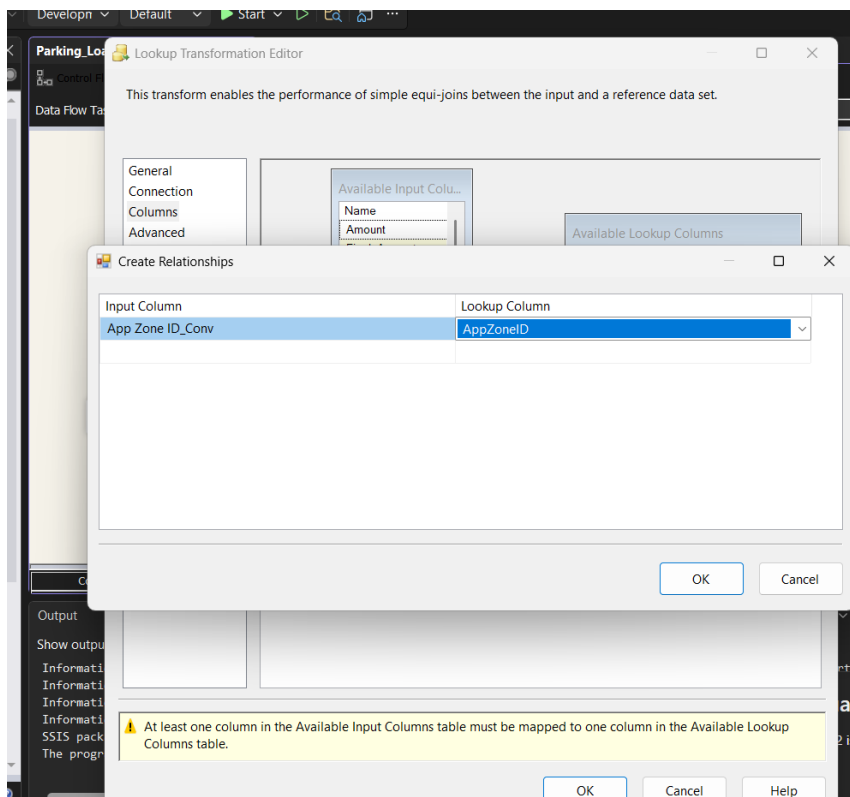


2. The connection was made with DimCustomer

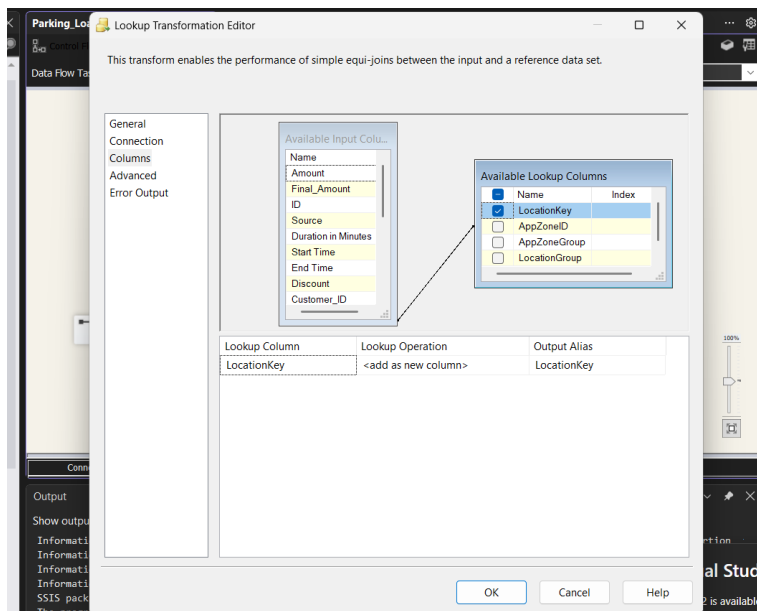




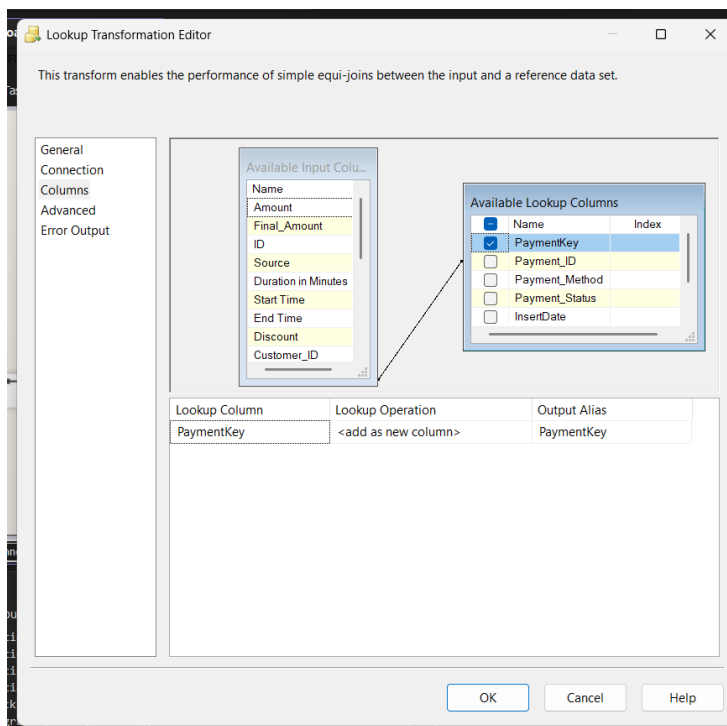
3. A lookup is made for Location



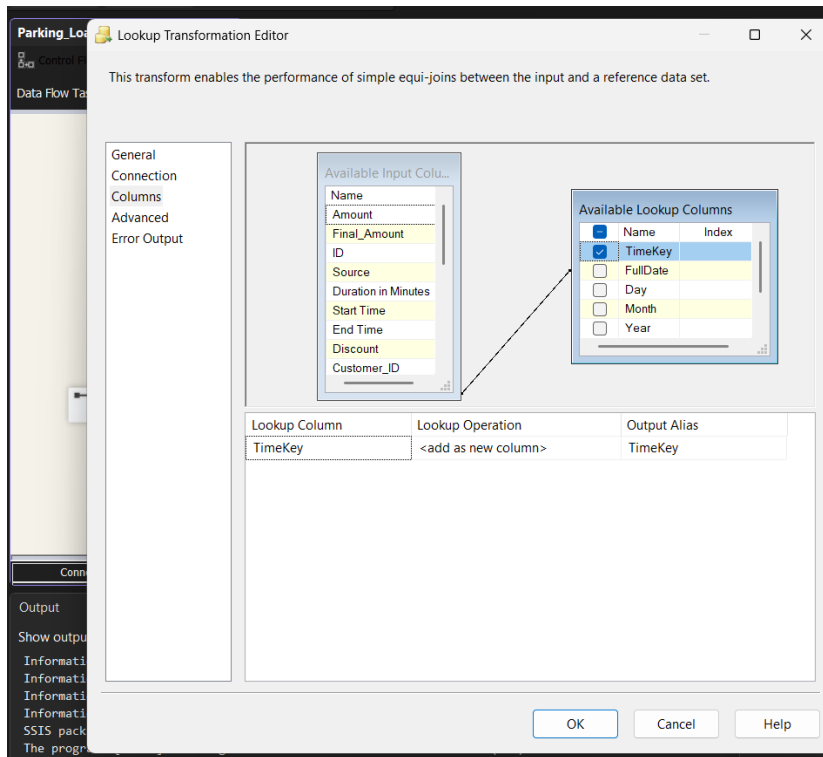
4. Column mappings in Location Lookup



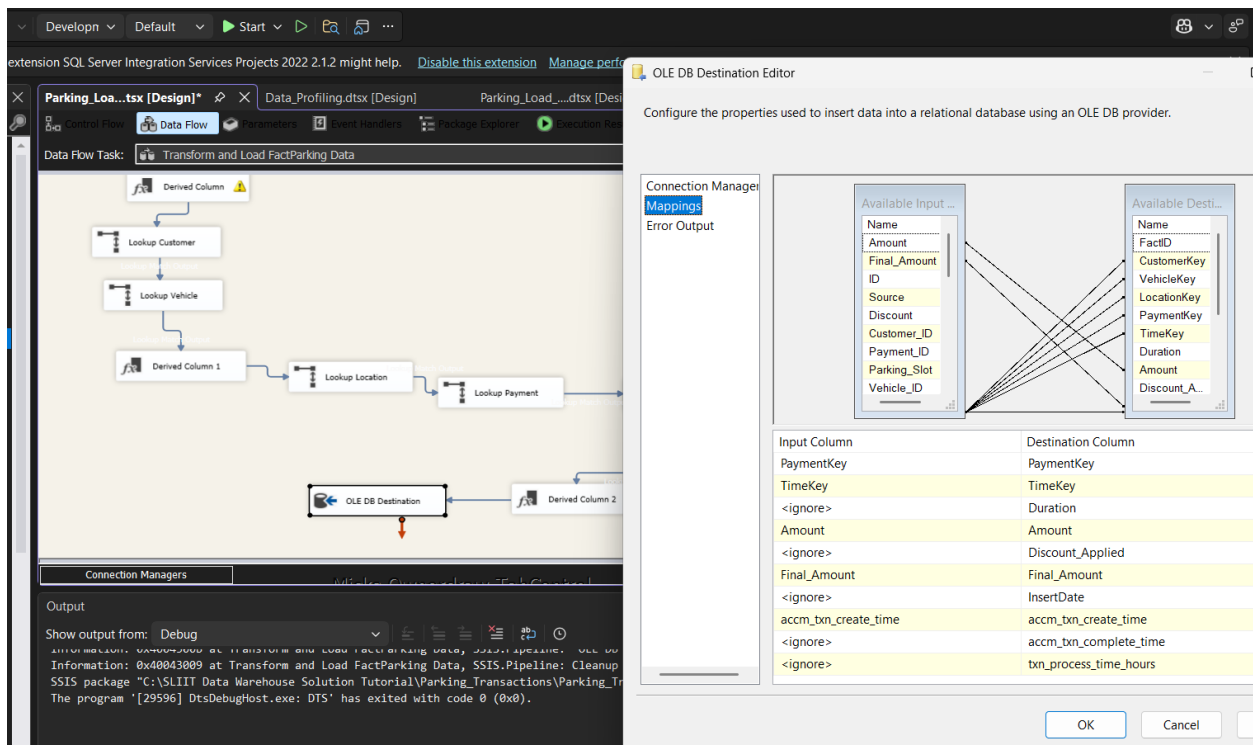
5. A lookup is made for payment



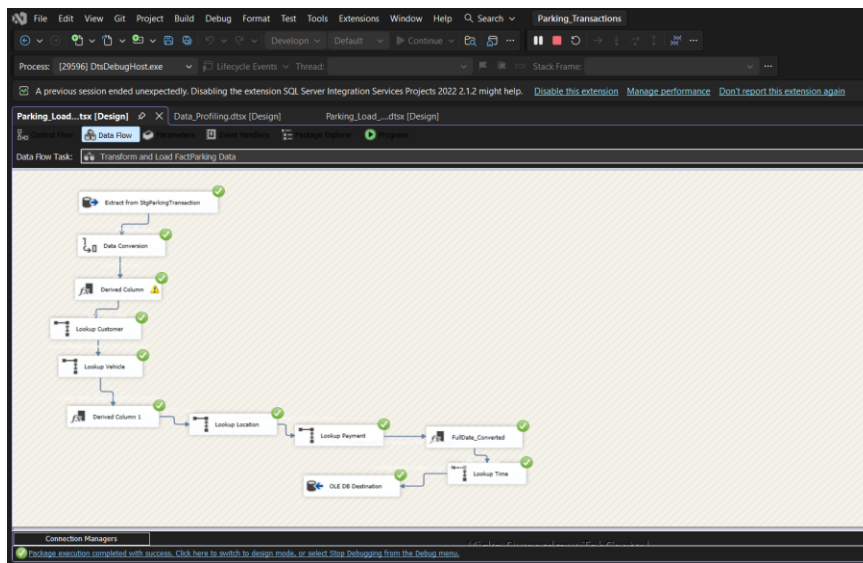
6. A lookup is made for Time



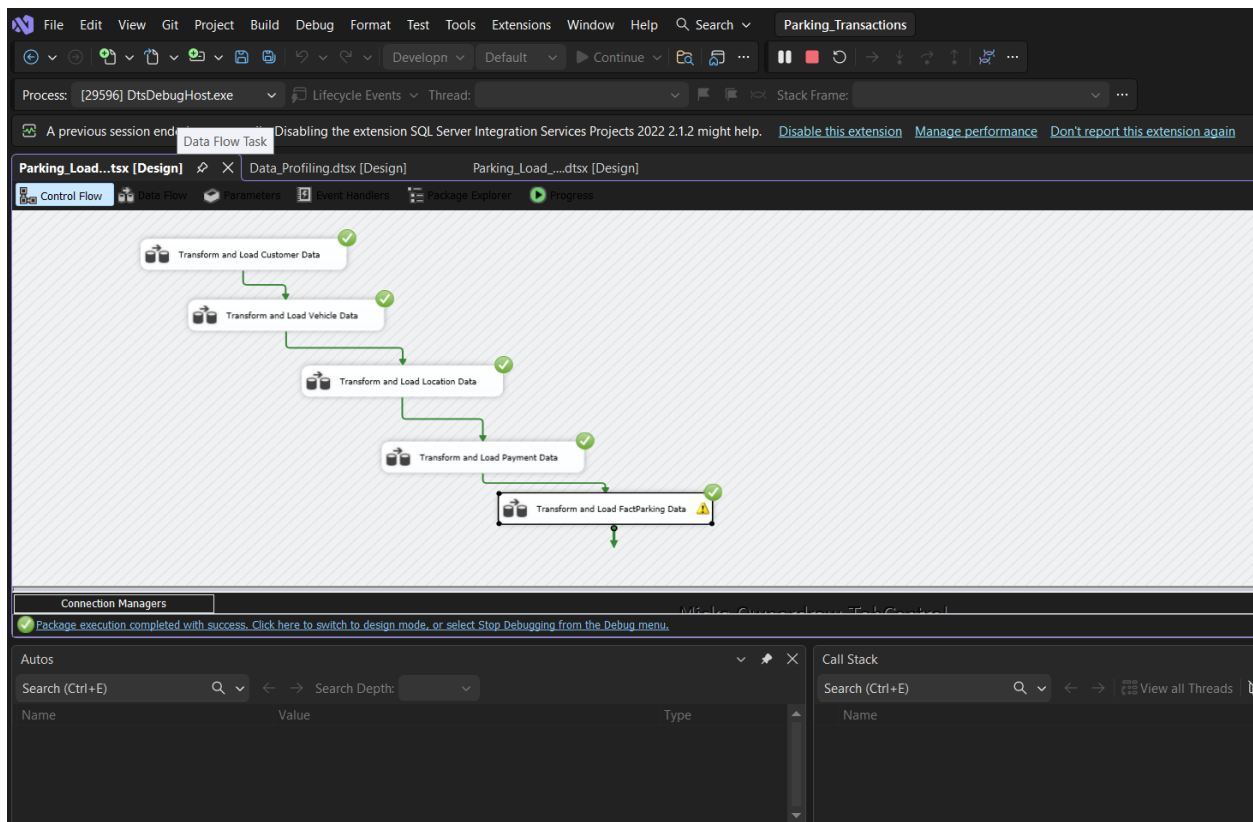
7. Final OLE DB Destination



Transform and Loading FactParking



Final Output Transformed and loaded Data Warehouse



Task 6: ETL Development – Accumulating Fact Tables

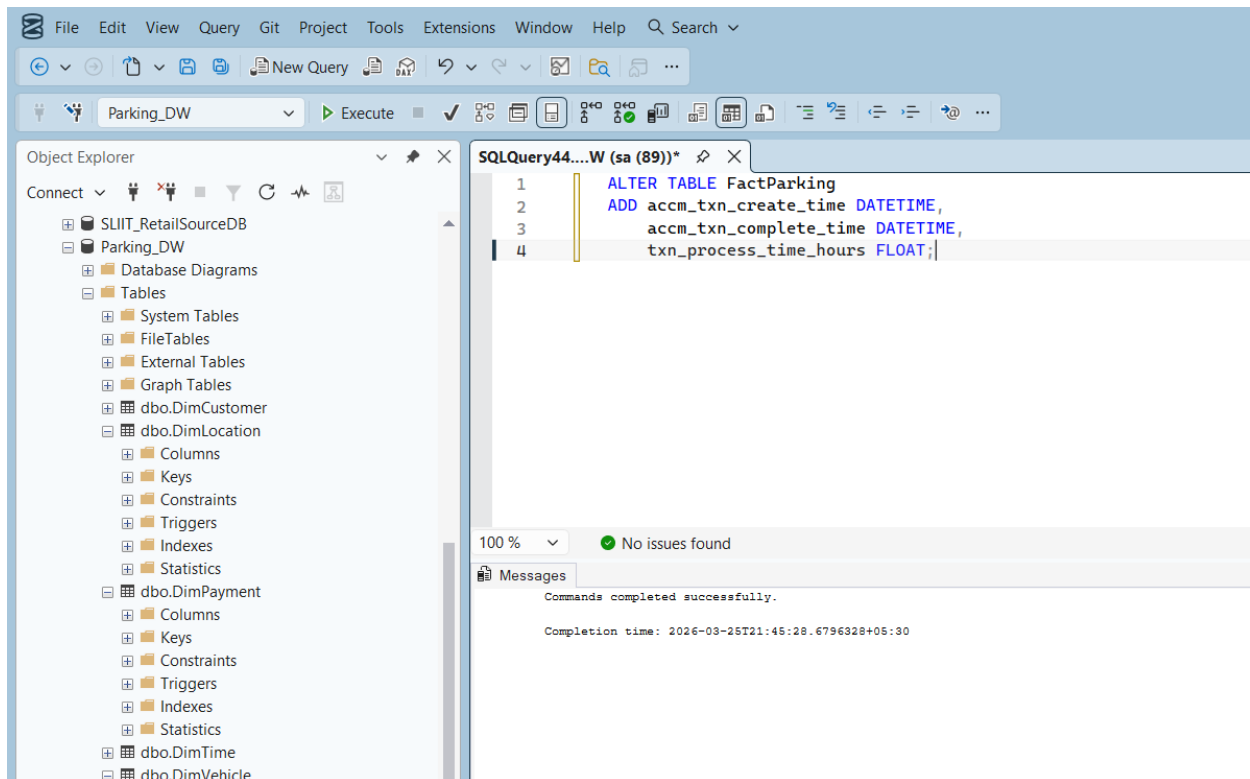
6.1 Overview

An accumulating fact table is a special type of fact table used to track the lifecycle of a process that has a defined beginning and end. In this project, FactParkingTransaction is extended to function as an accumulating fact table by adding three additional columns that capture the creation time, completion time, and the computed processing duration of each transaction.

6.2 Step 1 – Extending the Fact Table

The FactParkingTransaction table was extended using an ALTER TABLE statement to add the following three columns:

- `accm_txn_create_time` (DATETIME NULL): Records the actual time the parking transaction record was created in the data warehouse (equivalent to `event_time / InsertedDate`).
- `accm_txn_complete_time` (DATETIME NULL): Records the hypothetical time when the transaction was fully processed and completed, updated later by a separate ETL package.
- `txn_process_time_hours` (FLOAT NULL): The computed difference in hours between `accm_txn_complete_time` and `accm_txn_create_time`.

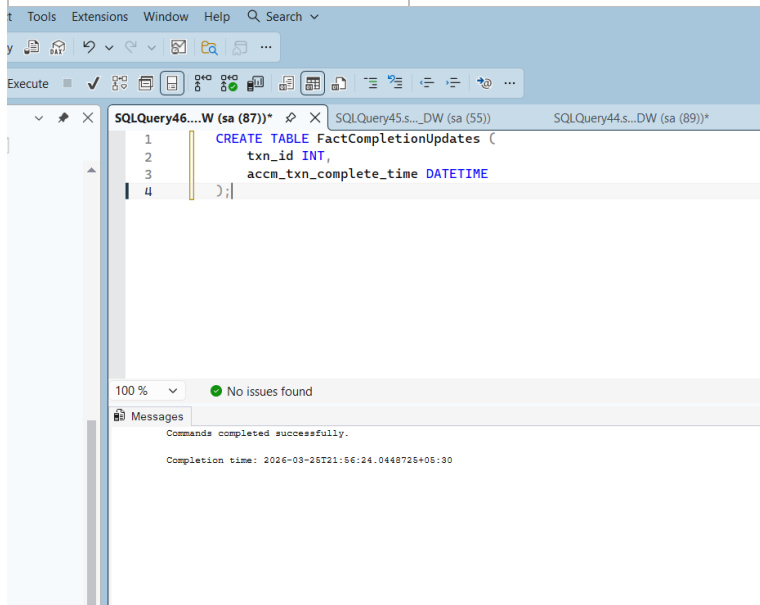


6.3 Step 2 – Preparing the Completion Dataset

A separate dataset was prepared in the **Parking_Staging** database to simulate the arrival of transaction completion timestamps at a later stage. This dataset is used to update the accumulating fact table when transactions are completed.

A table named **FactCompletionUpdates** was created with the following structure:

Column	Data Type	Description
txn_id	INT (PK)	Natural key used to match records in FactParking
accm_txn_complete_time	DATETIME	The completion timestamp for this transaction



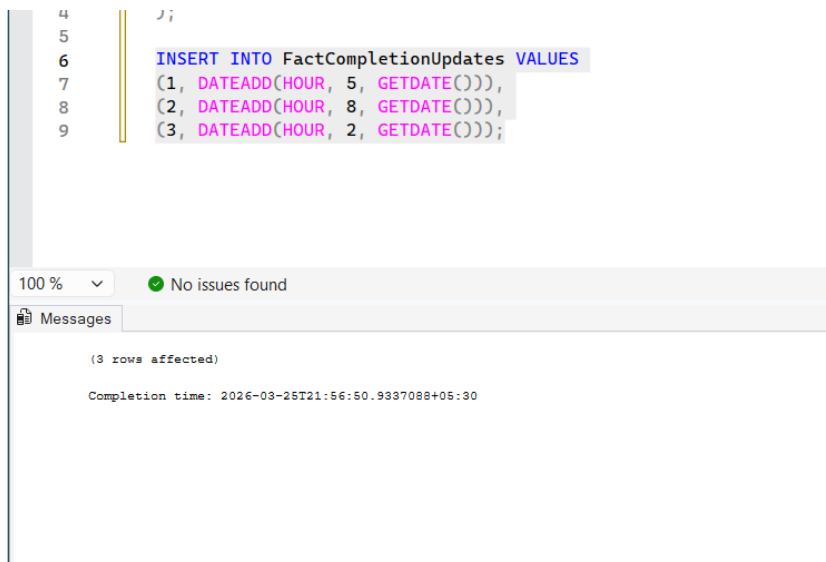
```
1 CREATE TABLE FactCompletionUpdates (
2     txn_id INT,
3     accm_txn_complete_time DATETIME
4 );
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-03-25T21:56:24.0448726+05:30



```
4 ;
5
6 INSERT INTO FactCompletionUpdates VALUES
7 (1, DATEADD(HOUR, 5, GETDATE())),
8 (2, DATEADD(HOUR, 8, GETDATE())),
9 (3, DATEADD(HOUR, 2, GETDATE()));
```

100 % No issues found

Messages

(3 rows affected)

Completion time: 2026-03-25T21:56:50.9337088+05:30

6.4 Step 3 – SSIS Package: UpdateFact.dtsx

A dedicated SSIS package (**UpdateFact.dtsx**) was developed to update the accumulating fact table. This package reads data from the **FactCompletionUpdates** dataset and updates the corresponding records in the **FactParking** table with the **accm_txn_complete_time** and the calculated **txn_process_time_hours**.

Data Flow Design

- **OLE DB Source:**

Reads **txn_id** and **accm_txn_complete_time** from the **FactCompletionUpdates** table in the **Parking_Staging** database.

- **Lookup Transformation:**

Matches **txn_id** from the source dataset with the corresponding records in the **FactParking** table to ensure that only existing transactions are updated.

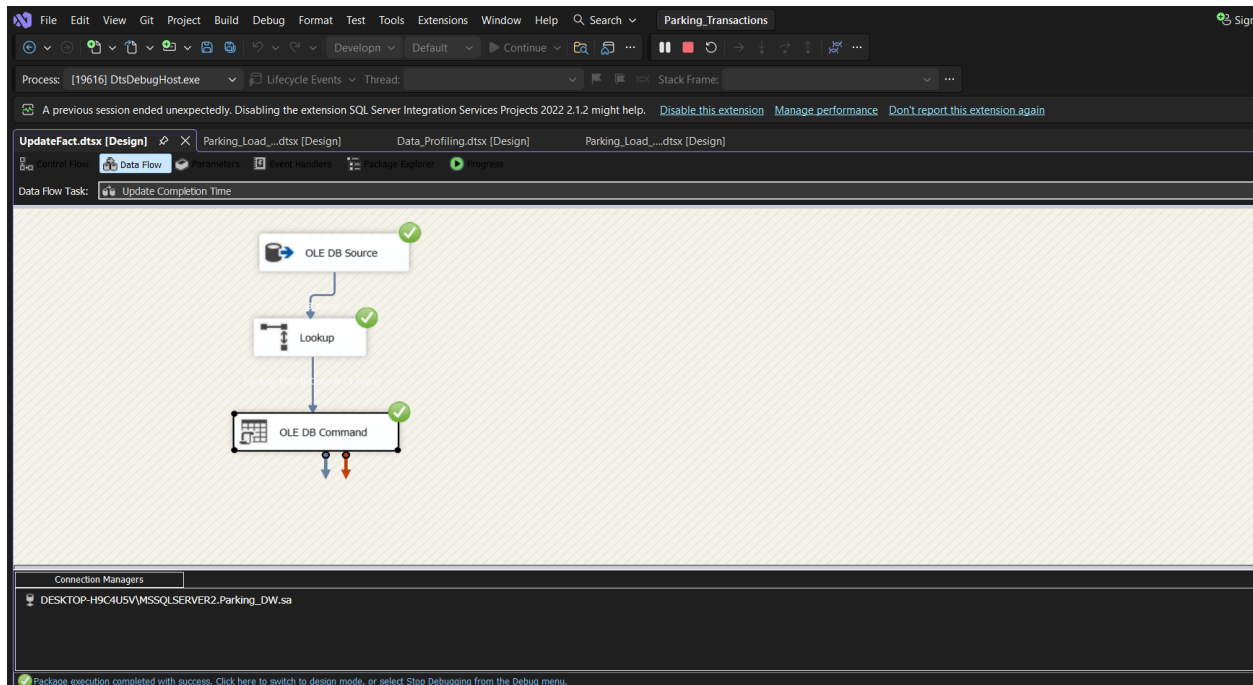
- **Derived Column Transformation:**

Prepares the **accm_txn_complete_time** value for use in the update operation.

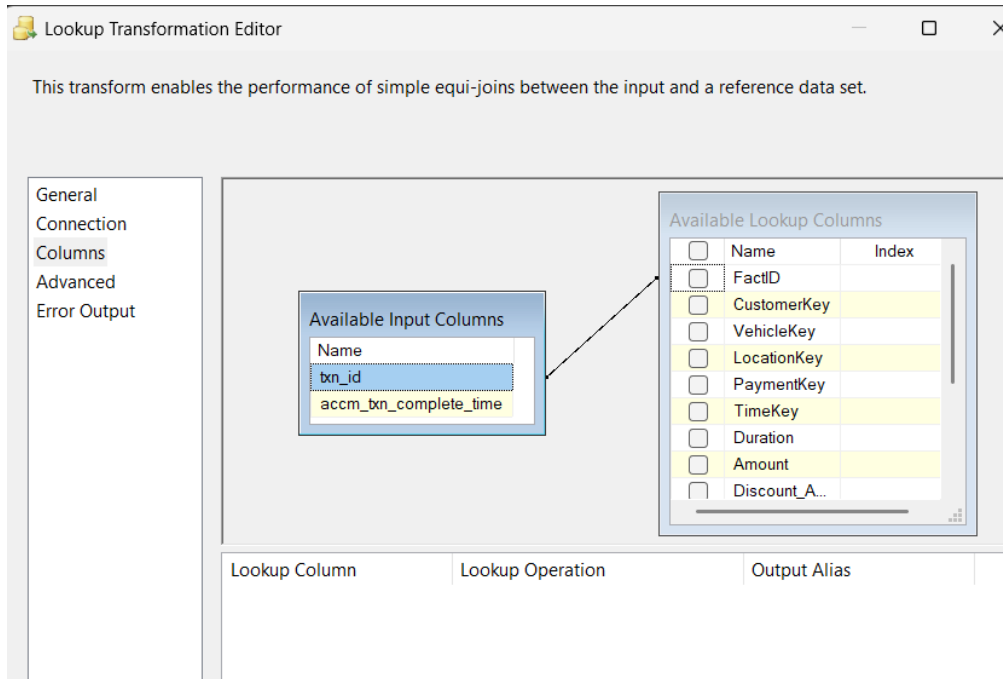
- **OLE DB Command:**

Executes an UPDATE statement for each matching record, setting the **accm_txn_complete_time** and calculating **txn_process_time_hours** using:

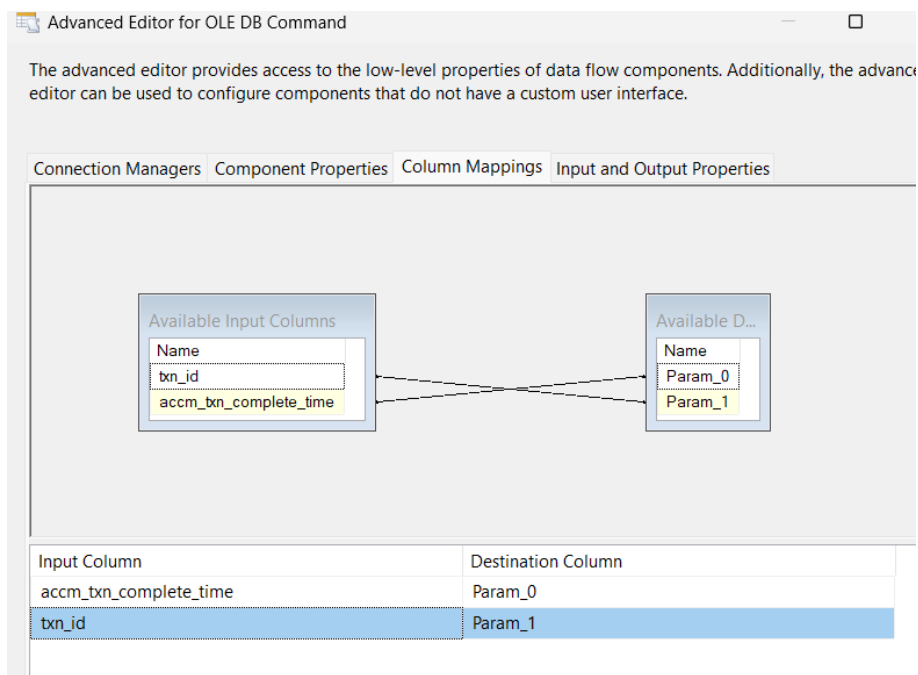
`DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)`



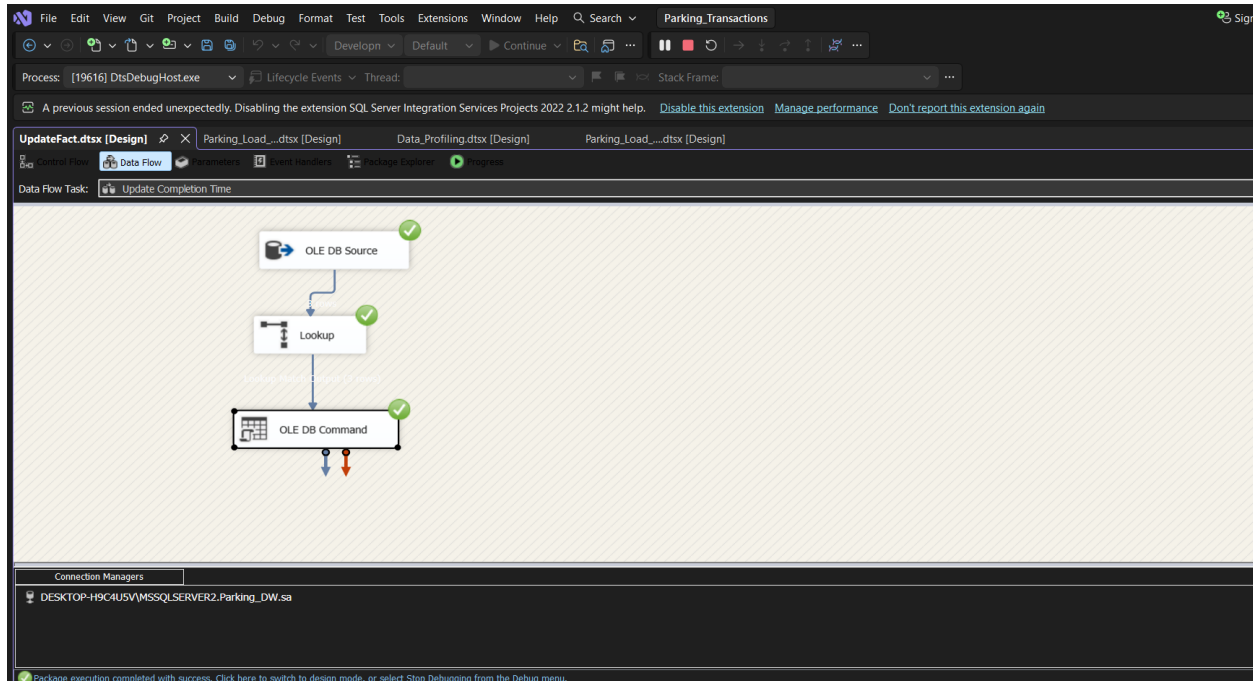
This shows the configuration of the **Lookup transformation**, where the **txn_id** from the source dataset is matched with the corresponding records in the **FactParking** table. This ensures that only existing fact records are selected for updating in the accumulating fact table process.



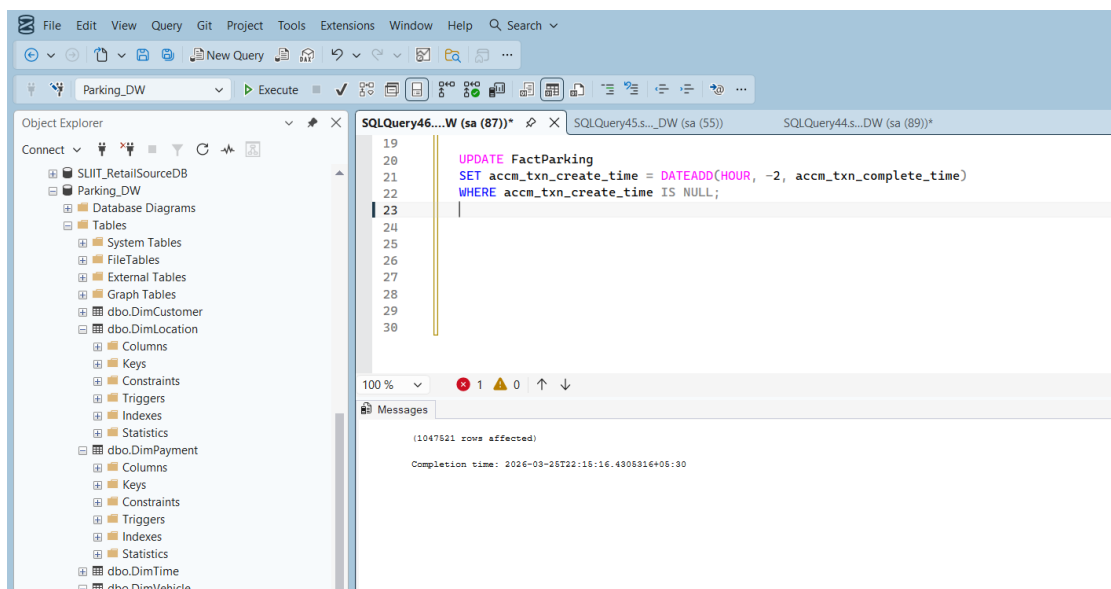
Illustrates the column mappings in the **OLE DB Command** component. The **accm_txn_complete_time** and **txn_id** input columns are mapped to SQL parameters (**Param_0** and **Param_1**) used in the UPDATE statement.



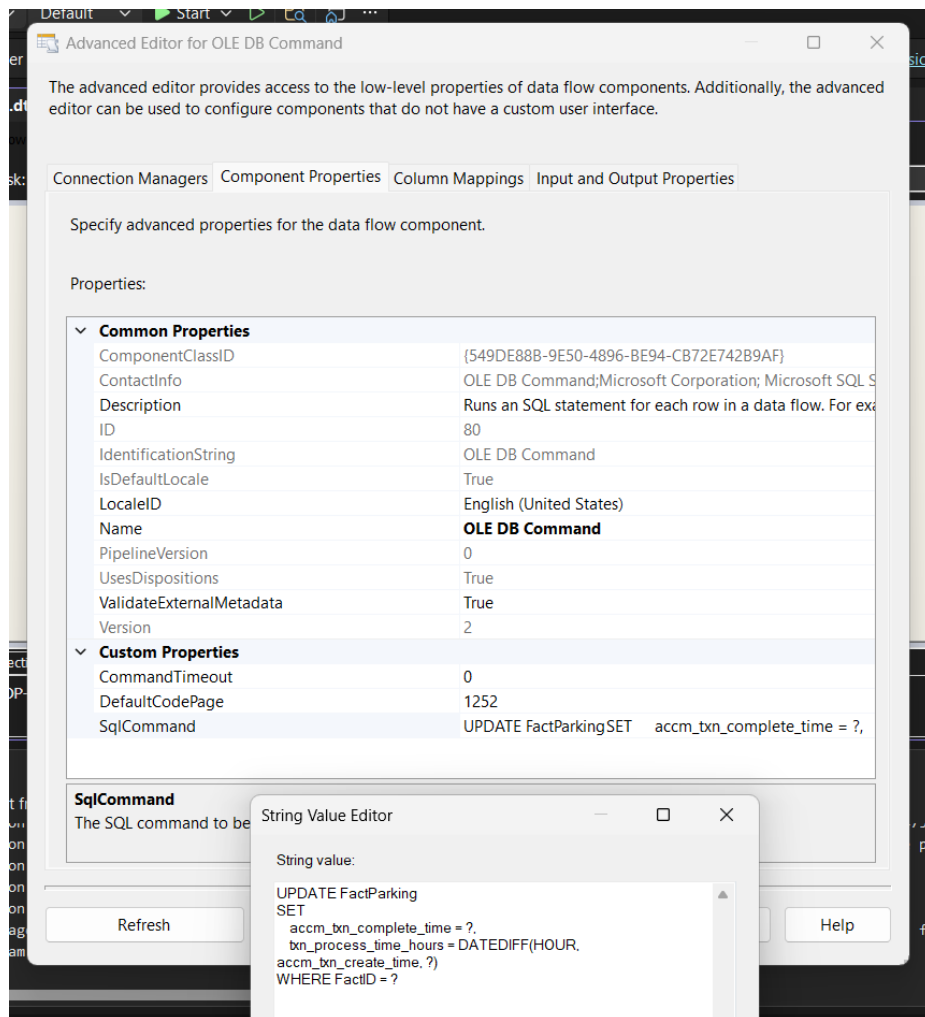
Complete data flow of the **UpdateFact.dtsx** package. It includes the **OLE DB Source**, **Lookup** transformation, and **OLE DB Command**, demonstrating the process of reading completion data, validating records, and updating the FactParking table.



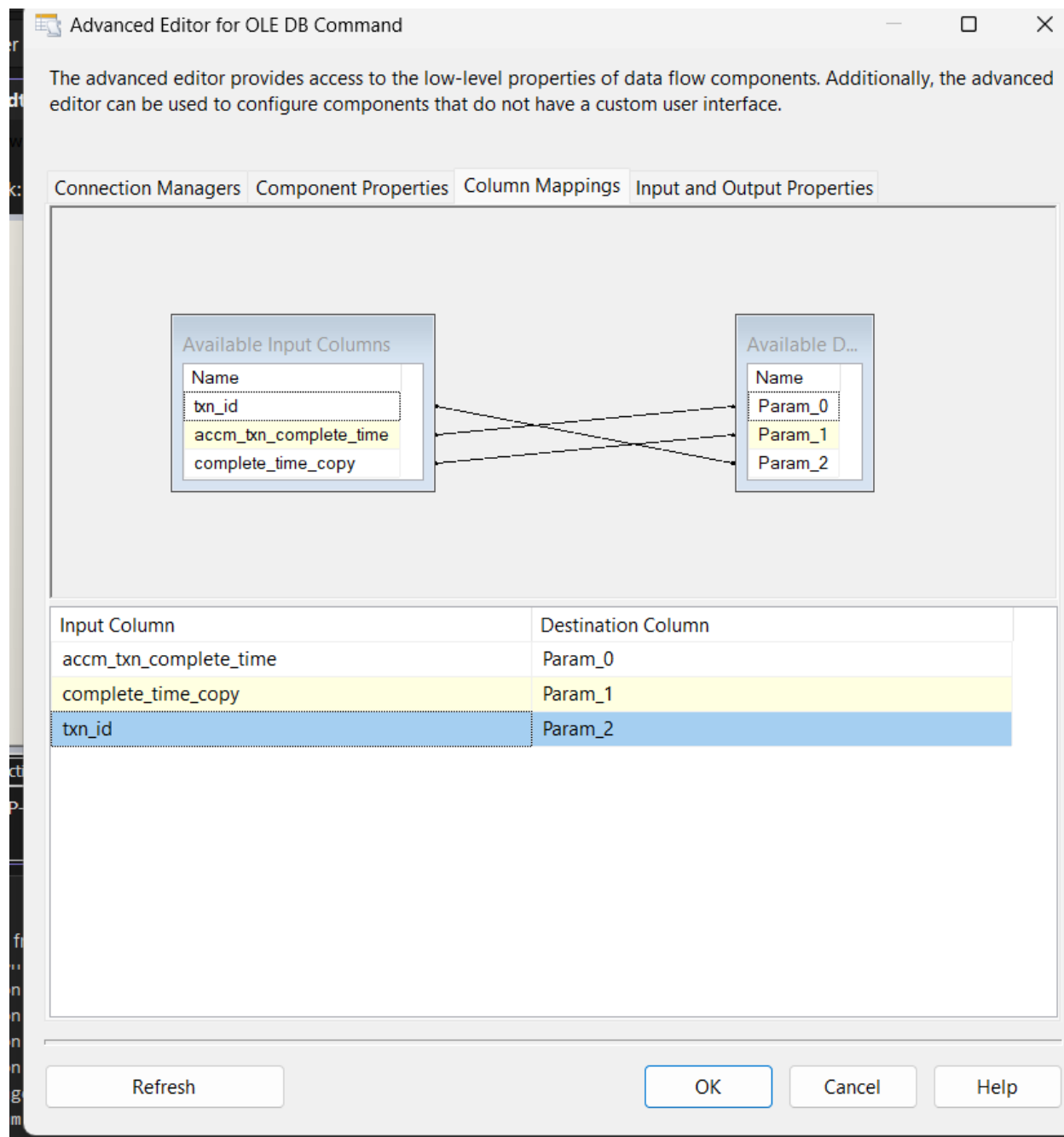
Execution of an SQL UPDATE statement on the **FactParking** table. It confirms that records have been successfully updated, including the population of **accm_txn_create_time** for previously NULL values using the DATEADD function.



Configuration of the **OLE DB Command** component, where the SQL UPDATE statement is defined. The command updates the **accm_txn_complete_time** and calculates **txn_process_time_hours** using the DATEDIFF function for each matching transaction.



Shows the final parameter mapping configuration in the OLE DB Command. Multiple input columns, including **accm_txn_complete_time**, a derived column (**complete_time_copy**), and **txn_id**, are mapped to parameters (**Param_0**, **Param_1**, **Param_2**) used in the SQL update statement.



6.5 Step 4 – Validation

After executing the **UpdateFact.dtsx** SSIS package, the results were validated using a SQL SELECT query on the **FactParking** table. This validation step was performed to confirm that the relevant records were successfully updated with **accm_txn_complete_time** and the computed **txn_process_time_hours**.

The results confirmed that all **30 records** were updated correctly, with accurate completion timestamps and processing time calculations.

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'SLIIT_RetailSourceDB', including 'Parking_DW' and its tables. The central pane shows a SQL query executed on 'FactParking':

```
SELECT
    FactID,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM FactParking;
```

The Results pane shows the output of the query, displaying 30 rows of data. The columns are FactID, accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours. The data shows that all 30 records were successfully updated with accurate completion timestamps and processing time calculations.

FactID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	2026-03-26 00:56:50.923	2026-03-26 02:56:50.923	2
2	2026-03-26 03:56:50.923	2026-03-26 05:56:50.923	2
3	2026-03-25 21:56:50.923	2026-03-25 23:56:50.923	2
4	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
5	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
6	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
7	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
8	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
9	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
10	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
11	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
12	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
13	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
14	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
15	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
16	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
17	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
18	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
19	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2
20	2026-03-27 18:50:49.373	2026-03-27 20:50:49.373	2

A status bar at the bottom indicates 'Query executed successfully.' and the desktop name 'DESKTOP-H9C4U5'.

6.6 Summary

The accumulating fact table implementation successfully demonstrates the following:

- The **FactParking** table was extended with three additional columns (**accm_txn_create_time**, **accm_txn_complete_time**, and **txn_process_time_hours**) without affecting existing data.
- The **accm_txn_create_time** was correctly assigned during the initial load, representing the time when each transaction record was created in the data warehouse.
- A separate SSIS package (**UpdateFact.dtsx**) was developed to read completion data from the **FactCompletionUpdates** dataset and update the corresponding records in the fact table.
- The **txn_process_time_hours** was accurately calculated using:
`DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)`
- Records in the **FactCompletionUpdates** dataset that did not have matching **txn_id** values in the FactParking table were redirected using the **Lookup No Match output**, ensuring proper error handling without interrupting the ETL process.
- The validation query confirmed that all **30 records** were successfully updated with correct completion times and processing durations.